CIM for Deep Learning — Step-by-Step Runbook (v4: model first, ASIC milestone)
- What changed in v4
- How to use this document
- No claims until there is data
- One design, three targets
- Diagrams
- The map
- STAGE 0 — Setup and resources (Weeks 1–2)
- STAGE 1 — The complete model in software (Weeks 2–10) ← START HERE
- STAGE 2 — Engine architecture on paper and in Python (Weeks 7–11)
- STAGE 3 — RTL of the complete engine, in simulation (Weeks 10–16)
- STAGE 4 — FPGA implementation, boards and demo (Weeks 5–18)
- STAGE 5 — ASIC implementation (Weeks 3–19)
- STAGE 6 — HLS of the complete model (Weeks 14–18)
- STAGE 7 — Measurement, then claims (Weeks 13–19)
- STAGE 8 — Demo polish, report, defence (Weeks 6–20)
- The critical path, at a glance
- Effort check
- Decisions to record
- Which LLM for what
- Data sources (verify before citing)
- Step numbering, old to new
- Monday morning (Week 3, from Monday 26 Oct 2026)

CIM for Deep Learning — Step-by-Step Runbook (v4: model first, ASIC milestone)

Project: 2023-FYDP-25 · UET Lahore · Supervisor: Prof. Dr. Muhammad Tahir

Calendar assumption: Week 1 started Monday 12 Oct 2026. Planned work finishes in Week 20 (ending Sunday 28 Feb 2027), including the report and defence. Weeks 21–24 (to Sunday 28 Mar 2027) are held as buffer: end of March is the hard limit. Mark exam and holiday weeks in the tracker and slide the plan around them. Estimates assume about 25 hours per person per week, and one effort day means 6 focused hours.

What changed in v4

Change	Why
The complete model is finished in software before any engine RTL is written. Stage 1 now covers data, training, quantization, the integer reference and frozen test vectors for the whole network. Hardware then builds the complete engine for that frozen model. The old "one layer on one tile, then scale up" path (old Steps 1.6–1.11 and 3.5) is gone.	The hardware is built once, for the model it will actually run. No layer-1-only engine, board test or synthesis run that later gets reworked.
ASIC is a full milestone (Stage 5, Milestone D), not a stretch step. It covers a standard-cell digital CIM tile and engine, the baseline at the same process node, place-and-route, power, a scaling study, and a custom digital SRAM-CIM macro.	It answers "but it's just an FPGA", gives area and energy numbers an FPGA cannot, and lets the design scale past FPGA resource limits.
The dataset is final (Step 0.4): four caliber-based firearm classes plus background, from the Kabealo, C3GD and Cadre field datasets and four background sources. See `docs/dataset_audit.md`.	Per-model classes cannot be split by session; caliber classes have 4–6 sessions each.
HLS covers the complete model (Stage 6), not layer 1 only.	Same reason as the first row.
Stage 0 and Step 1.2 are done and unchanged. Step numbers in the other stages have changed; the mapping is in the appendix "Step numbering, old to new". `docs/arith_spec.md` and `docs/decisions.md` were written against v3 and cite v3 step numbers (for example "Step 2.6" for quantization, now 1.10); read them through the same table.	

How to use this document

Every step has the same shape:

- **Goal:** what this step proves.
- **Do:** the actions, in order.
- **Output:** the exact files or artifacts that must exist when you finish.
- **Review gate:** a checklist that someone other than the author runs. Every box must pass before the next step starts.
- **If it fails:** the first thing to try.
- **LLM help:** which model does what for this step.

The line under each step heading gives the calendar window (weeks), the effort in person-days (one day = 6 focused hours, so "10 days" may be two people for a week), and what the step needs first.

The one rule: a step is not done because the author says so. It is done when the reviewer ticks every box in the review gate and shows the output in the weekly meeting.

The second rule: every part is tested on its own the day it is finished: each Python function against the spec, each RTL block against the Python reference, each netlist against the RTL.

Steps are numbered by stage. Within a stage, do them in order. Where two stages can run at the same time (you have four people), the "Lane" note says so. Who takes which lane is your call.

No claims until there is data

Nothing in this project is claimed until it has been measured. That gives two kinds of statement, and they must never be mixed up.

- **Design targets.** Things you build to. You control them, and they are either met or not met. Examples: "bit-exact with the integer reference," "zero DSP slices in the CIM tile array," "≥ 8 MACs per cycle," "clock ≥ 50 MHz," "the SRAM-CIM bitcell array passes DRC and LVS."
- **Hypotheses.** Things you test, and you don't know the answer yet. Example: "the CIM engine's latency scales better with MAC count than an equivalently provisioned conventional engine." Stage 7 measures it. The claim is written after that, in words the data supports.

ASIC numbers are tool estimates, never measurements. Nothing is fabricated. Area comes from place-and-route, clock frequency from static timing analysis at a stated corner, power and energy from the power tool driven by switching activity from gate-level simulation. Every ASIC number in the report says which tool, which stage (post-synthesis or post-layout), which corner and which activity source it came from, and is called an estimate.

No speedup, area or energy ratio belongs anywhere in the synopsis, the slides or the report until Step 7.4 has produced it.

One design, three targets

The same RTL in `rtl/core/` goes to three targets. Board wrappers and ASIC wrappers stay thin.

	Nexys A7-100T	**ZCU104**	**ASIC (standard cells + custom SRAM-CIM)**
Part / library	`xc7a100tcsg324-1`	`xczu7ev-ffvc1156-2-e`	Cadence generic 45 nm PDK (GPDK045) or the PDK confirmed in Step 0.1
Family	Artix-7 (FPGA only)	Zynq UltraScale+ MPSoC (FPGA fabric plus Arm processors)	Standard-cell logic; custom bitcell array for SRAM-CIM
LUTs	63,400	230,400	— (area in µm²)
Block RAM	4,860 Kb	312 × 36 Kb, plus 96 UltraRAM	No compiled memory used; weights in flip-flops, latches or the custom SRAM-CIM macro
DSP slices	240	1,728	—
Memory on board	128 MiB DDR2	2 GB DDR4 on the processor side	—
Pmod ports	4	3	—
Microphone	onboard PDM microphone	none	—
Role	Development board	Final FPGA platform and live demo	Area, clock and energy at a process node; scaling beyond FPGA limits

(Nexys figures from Digilent's Nexys A7 page; ZCU104 figures from the AMD/Xilinx kit listings. Confirm both against the datasheets before you size anything.)

What CIM means on each target.

- **FPGA:** a CIM *emulator*. Weights sit in LUTRAM fused to the AND mask and adder trees that consume them. The dataflow is that of an all-digital SRAM-CIM macro, but the storage is general-purpose fabric.
- **ASIC, standard cells:** a *synthesizable digital CIM tile*. Weight bits live in flip-flops or latches placed next to their AND gates and the adder tree. It is real compute-in-memory dataflow in silicon, with register-file storage. Name it that way in the report; never call it SRAM-CIM.
- **ASIC, custom macro:** a *digital SRAM-CIM macro*. Each bitcell is 6T SRAM storage plus a NOR/AND gate, so the 1-bit product `weight_bit AND activation_bit` is formed inside the array. The 8 cells of one weight output either `w` or `0`, which is exactly the gated weight the tile's adder tree already takes. The periphery (adder tree, shift-accumulate, MSB subtract, requantization) stays standard cells, synthesized from the same RTL.

Five rules that keep three targets from tripling the work:

1. **The core is target-neutral.** `rtl/core/` (tile, engine, baseline, requantization, counters) contains no vendor primitives, no FPGA-only attributes and no `initial` blocks. Weight storage goes through one wrapper, `weight_store`, with a `STORAGE` parameter (`"lutram"`, `"ff"`, `"latch"`, `"macro"`). Board and ASIC specifics live in `rtl/board/nexys/`, `rtl/board/zcu104/` and `rtl/asic/`. The rules are written down in Step 2.5.
2. **Design to the smaller board first.** If the model and engine fit the Nexys, they fit the ZCU104. The ZCU104 and the ASIC then extend the sweep to larger MAC counts; they do not change the model.
3. **Never mix numbers from different targets in one comparison.** Cycle counts are the same everywhere, because they come from the design. Clock frequency, LUTs, area and power are not. FPGA rows go in `results/runs.csv` (with a `board` column); ASIC rows go in `results/asic_runs.csv`. Every figure names its target.
4. **The FPGA measurement platform is a decision, not an assumption** (Decision D14), made once the complete engine has run on both boards (Step 4.4).
5. **The ASIC compares like with like.** CIM and baseline are synthesized to the same library, at the same clock target, with the same storage cell type, so the only difference left is the dataflow.

Optional, not scheduled: a DRAM-bound baseline on the ZCU104. At this model size all weights fit on chip, so no real memory wall exists. The ZCU104's processor-side DDR4 would let a second baseline fetch weights from off-chip memory. It needs the processor-to-fabric data path and is real extra work. The runbook is complete without it.

Diagrams

Diagrams are review tools, not decoration. A datapath drawn before the RTL gives the reviewer something to check the code against, and it is what you will read when a testbench fails at 1 a.m. Every diagram below is the output of a step, and each is checked by someone who did not draw it.

Diagram	Drawn in	Must show
Verification ladder	Step 1.2 (done); ASIC rungs added in Step 2.5	Float → integer reference → Python engine model → RTL sim → FPGA post-synthesis → board; RTL → Genus netlist (LEC + gate-level sim) → post-layout sim with SDF. The pass criterion on each rung
Layer-to-tile mapping	Step 2.1	Each conv layer split into 16-row chunks, which tile and slot holds which weights, ping-pong buffers

Diagram	Drawn in	Must show
CIM vs baseline, side by side	Step 2.3	The same MAC array in both; the only difference is where weights come from
Tile datapath	Step 3.1 (before the RTL)	Weight storage per row, the AND mask, the per-column adder tree, shift-accumulate with the MSB subtract, bit width on every wire, pipeline registers, the `BPC` adder trees
Bit-serial timing	Step 3.1 (before the RTL)	The 8 / BPC cycles, which bit-planes are broadcast when, when the accumulator is valid
Engine block diagram and controller state machine	Step 3.2	Tiles, buffers, window addressing for every layer, requantization, pooling, GAP, dense, argmax, the layer/position/chunk loops, cycles per state
System block diagram	Step 2.1 (draft), Step 4.8 (final)	Microphone, I²S receiver, event detector, ring buffer, engine, host link, clock-domain crossing marked, both board wrappers
ASIC flow	Step 5.1	RTL → Genus → Conformal → Innovus → Quantus/STA → Joules, and the custom path Virtuoso → Spectre → PVS → Quantus → LEF/.lib
SRAM-CIM bitcell and array	Steps 5.2, 5.3	Transistor schematic of the compute bitcell, array floorplan, compute wordlines, write path, outputs to the adder tree

The side-by-side CIM-versus-baseline diagram is the most important one for your viva. It makes the fairness argument visible in a way no paragraph does.

How to work.

1. Sketch first, then write RTL, then redraw. The clean version is redrawn after the tests pass so it matches the code exactly. A diagram that disagrees with the RTL is worse than no diagram.
2. Every block is a module, and every module is a block. This is a review check, run by a non-author.
3. Conventions: bit widths on every wire; registers drawn differently from combinational logic; clock domains in different colours; signal names identical to the RTL.
4. Tools: draw.io (free). Keep the editable `.drawio` file in `docs/diagrams/` and export SVG or PDF, never screenshots. WaveDrom for timing diagrams.
5. LLMs draft, you verify. A model will produce draw.io XML, Mermaid or WaveDrom that looks right and has wrong bit widths or a missing MSB subtract. Check every model-drawn diagram against `arith_spec.md`.

The map

```
STAGE 0   Setup & resources                          Weeks 1–2     done
STAGE 1   The complete model in software             Weeks 2–10    ← START HERE   → Milestone A (Week
10)
STAGE 2   Engine architecture, paper + Python        Weeks 7–11
STAGE 3   RTL of the complete engine (sim)           Weeks 10–16
STAGE 4   FPGA, boards and demo                      Weeks 5–18    (4.1 board link is an early lane)
                                                                   → Milestone B (Week 15), Milestone
C (Week 18)
STAGE 5   ASIC                                       Weeks 3–19    (5.1–5.3 are early lanes) →
Milestone D (Week 19)
STAGE 6   HLS of the complete model                  Weeks 14–18
STAGE 7   Measurement → claims                       Weeks 13–19
STAGE 8   Demo polish, report, defence               Weeks 6–20    then Weeks 21–24 = buffer
```

Why the model comes first

The old plan pushed one real layer through the whole chain by Week 8 and then added layers one by one. That found interface bugs early, but it built hardware twice: a layer-1 engine, a layer-1 synthesis and a layer-1 board run, all reworked when the other layers arrived, and every model change rippled into RTL that was already on the board.

This plan finishes the model first and builds the hardware once:

1. **Stage 1 freezes the complete model:** architecture, trained weights, quantization, the integer reference for every layer, and per-layer test vectors. Its accuracy is known before any RTL exists (Milestone A).
2. **Stage 2 freezes the engine on paper and in Python:** the mapping spec, the baseline spec, the rules for target-neutral RTL, and a Python model of the complete engine that is bit-exact with the integer reference. It is the golden model the RTL is checked against, block by block.
3. **Stage 3 writes the RTL for the complete engine** and integrates it in simulation, layer by layer, against that golden model. The one-at-a-time discipline stays, but in simulation, where adding a layer costs minutes rather than a synthesis and board run.
4. **Stages 4 and 5 take the finished engine to the boards and to the ASIC flow.**

Three things stop this from becoming "everything meets for the first time in month 6":

- **Every interface is fixed in Stages 1 and 2:** `arith_spec.md`, `file_formats.md`, `mapping_spec.md`, the UART protocol, and the Python engine model.
- **Every block is unit-tested against Python the day it is written** (Step 3.2), before it is connected to anything.
- **The board link is built and tested early on its own** (Step 4.1, Weeks 5–8), so the complete engine goes onto the board the day it passes simulation.

Work that does not depend on the model runs in parallel lanes from the start: the board link (Step 4.1), the Cadence flow bring-up (Step 5.1) and the custom SRAM-CIM bitcell (Step 5.2). The weights cannot change a 6T + NOR bitcell or a UART, so these lanes are not "hardware before the model", they are infrastructure.

STAGE 0 — Setup and resources (Weeks 1–2)

> **Status: done.** The text below is unchanged from v3. Its outputs are `docs/resources.md`, the revised objectives, the repository skeleton, and `docs/dataset_audit.md` with `Dataset-Final/manifest.csv`. The ASIC design targets are added to the objectives by an addendum in Step 5.1, not by editing Step 0.2. Step numbers mentioned inside Stage 0 (for example "Step 1.11", "Stage 2", "Step 4.4") are v3 numbers; see "Step numbering, old to new" at the end.

Step 0.1 — Resource inventory with Sir

Week 1 · 1–2 days · Needs: nothing

Goal. Know exactly what hardware and tools you have before planning around them.

Do. Get written answers to these questions:

1. FPGA boards: confirm the Nexys A7-100T (`xc7a100tcsg324-1`) and the ZCU104 (`xczu7ev-ffvc1156-2-e`). For each: is it ours for the year, or shared in the lab? Can it leave the lab? When is the ZCU104 available to you?
2. Cadence: which tools exactly? Ask for the licence list. The ones that matter to you:
    - Xcelium (simulator)
    - JasperGold (formal verification)
    - Conformal (equivalence checking)
    - Genus (logic synthesis)
    - Innovus (place and route)
    - Stratus HLS (high-level synthesis)
    - Joules (power)
3. Cadence: which process design kit (PDK) and standard-cell library? For example Cadence's generic 45 nm (GPDK045) or a foundry kit through Europractice. Without a PDK, Genus and Innovus are useless to you. Is there an SRAM memory compiler?
4. Cadence access: lab machines only, or remote server? Hours? Is there anyone who has run the flow before?
5. Compute: is there a GPU machine, or is it Colab/Kaggle? (The models are small, so free Colab is enough.)
6. Microphone: budget approval for an I²S MEMS microphone (INMP441 or similar). The Nexys A7 has an onboard PDM microphone, but the ZCU104 has none, so an external I²S microphone on a Pmod is the one that works on both boards. Check the Pmod voltage in the ZCU104 board user guide (UG1267) before wiring anything.
7. University calendar: the dates of the mid-term evaluation, any progress report or poster, and the final report and defence. Can a revised synopsis be submitted formally, or do the changes from Step 0.2 go into the mid-term report? Put the dates in the tracker and line Milestone A up with the mid-term evaluation.
8. ZCU104 tooling: does the Vivado licence on the lab machines cover the ZU7EV device? (The kit is sold with a Vivado Design Edition seat, but confirm that your lab's copy came with it.) Is the board's power supply and USB cable present? Has anyone in the lab used the ZCU104 with PYNQ or Vitis?

Output. `docs/resources.md`, with every question answered and the date and source of each answer.

Review gate.

- [ ] Every one of the eight questions has an answer, or an explicit "unknown, asking by DATE."
- [ ] Both FPGA part numbers are copied exactly as Vivado expects them.
- [ ] The Cadence tool list says which tools are confirmed and which are assumed.

If it fails. Missing answers don't block Stage 1: it runs on the Nexys A7 or in pure simulation. A missing ZCU104 date or licence blocks only Step 1.11 and the final platform, so keep chasing. Everything still works on the Nexys alone, with the ZCU104 results as an extension.

LLM help. None needed.

Step 0.2 — Rewrite the objectives with no claims

Week 1 · 2 days · Needs: nothing

Goal. Objectives your supervisor signs off that promise only what you can guarantee.

Do. Replace Chapter 4 with something like the following. Adjust the wording, but keep the target/hypothesis split.

> **Design targets**
>
> 1. Assemble a five-class dataset (four firearm classes plus a background/reject class) from public sources, with train/test splits that share no recording session.
> 2. Train and compare a raw-waveform 1D-CNN and a spectrogram 2D-CNN within a fixed hardware budget (≤ 4 M MACs, and few enough parameters to fit the FPGA's distributed RAM, provisionally ≤ 30 k), and report accuracy on the session-disjoint test split.
> 3. Quantize the selected model to 8-bit integers (and explore 4–6 bits), and define an integer-only reference implementation.
> 4. Design a digital CIM macro emulator (weight-stationary, bit-serial, no hard multipliers in the tile array) delivering at least 8 MACs per clock cycle at a target clock of at least 50 MHz.
> 5. Achieve bit-exact agreement between the hardware and the integer reference on the full test set.
> 6. Implement the engine in hand-written RTL and in HLS, and compare resources, clock frequency, latency, initiation interval and design effort.
> 7. Demonstrate live classification from a microphone on an FPGA board.
>
> **Hypothesis to be tested**
>
> 8. Because each row of a weight-stationary CIM macro has private weight storage, its inference latency should keep improving as parallelism is added, while a conventional engine fetching weights from shared memory should saturate at its memory-port bandwidth. We will measure inference latency and initiation interval for both architectures across 8–128 parallel MAC units and report the observed scaling.

Also make these edits:

- **Abstract.** Remove the "5×" figure. Say what you will measure and compare.
- **Objective 8 (power and energy).** Drop it, or demote it to "vendor-estimated power reported for context."
- **SDG 7.** Soften. You are not measuring energy, so say the architecture removes repeated weight fetches, which is an energy-relevant property, without claiming a saving.
- **Accuracy target.** Don't promise 90% before you have seen Sir's dataset. As a reference point, the public Kabealo gunshot dataset's own technical validation (a spectrogram CNN) averaged about 89% across 100 trained models, and a session-disjoint split is a harder test than most published numbers. Use "report accuracy on a session-disjoint split" now, and agree a number with Sir after the Step 0.4 audit.
- **Methodology.** Add one paragraph: "On an FPGA, weights cannot be stored inside a multiplier-capable SRAM array. We implement a CIM macro emulator: weights held in distributed-RAM primitives fused to the arithmetic that consumes them, with the dataflow of an all-digital SRAM CIM

macro. The tile array (weight storage, AND mask, adder trees, shift-accumulate) uses no DSP slices. Requantization between layers multiplies by a per-channel scale and is reported separately."

- **Scope of the "no DSP" statement.** It applies to the tile array, not to the whole engine. Requantization multiplies a 32-bit accumulator by a 16-bit scale, which Vivado places in a DSP slice by default, and it sits outside the macro. Write the statement exactly that way, report DSP use per module, and give the whole-engine total next to it. The baseline engine uses the identical requantization unit, so the comparison stays fair. Optionally, add one variant with the requantization multiplier forced into logic, to show a fully DSP-free engine; it costs LUTs, so report it as a separate row. Record the final scope as Decision D17 and agree it with Sir.
- **Figures 5.1 and 5.2.** The current block diagram and flowchart are out of date: they feed the spectrogram path straight into live hardware, show no baseline, and show no microphone-to-board path. Note them for redrawing in Steps 3.1 and 4.4, and use the redrawn versions at the mid-term evaluation.
- **Technology target (Chapter 5.1) and costing (Chapter 8).** Name the Nexys A7-100T as the development board and the ZCU104 as the final platform, and describe the design as board-independent. If the boards are lab-provided, the board line in the costing changes; confirm with Sir.

Output. Revised Chapter 4, Abstract, SDG section, Chapter 5.1 and Chapter 8.

Review gate.

- ☐ Search the document for "×", "times", "reduction", "faster" and "lower". No comparative claim survives outside the hypothesis.
- ☐ Every design target is either met or not met (nothing vague like "efficient").
- ☐ Every "no DSP" statement says "tile array" and not "engine" or "design".
- ☐ Sir has seen and approved it.

LLM help. Opus drafts. Then paste the result into GPT with: "Find every sentence that claims a result we have not yet measured."

Step 0.3 — Repository, tools, build skeleton

Week 1–2 · 3 days · Needs: 0.1 (part number)

Goal. Every team member can clone, simulate and synthesize "hello world" from the command line.

Do.

1. Create a private GitHub repo with this layout:

```
sw/   tb/   hls/   host/   results/   docs/
rtl/core/              board-agnostic design
rtl/board/nexys/       Nexys A7 wrapper (clock, pins, UART)
rtl/board/zcu104/      ZCU104 wrapper
fpga/nexys/   fpga/zcu104/      constraints, TCL, reports
docs/diagrams/         .drawio sources plus exported .svg
```

2. Set up the Python environment: `requirements.txt` with numpy, tensorflow, scikit-learn, librosa, soundfile, matplotlib, pandas, cocotb, pyserial.

3. Install the simulator: Icarus Verilog or Verilator, plus cocotb and GTKWave. (Use Xcelium later if Cadence is confirmed.)

4. Install Vivado ML Standard (free; it includes Vitis HLS) on at least two machines. During installation, select device support for both Artix-7 and Zynq UltraScale+ MPSoC. Confirm in the tool that the ZU7EV part is available to you; if it needs a licence, see Step 0.1 question 8.

5. Write `fpga/tcl/build.tcl`. It takes a top module, a board name and the matching part number, then runs synthesis → implementation → bitstream, and writes the utilization and timing reports to `fpga/reports/<config>/`. No GUI.

6. Write a Makefile with targets `sim`, `synth` and `bit`, each taking `BOARD=nexys` or `BOARD=zcu104`.

7. Create `results/runs.csv` with the header: `date,commit,board,stage,config,n_macs,f_clk_mhz,cycles,ii,lut,ff,bram,dsp,dsp_array,acc,bitexact_pct,notes` (`dsp` is the whole-design total; `dsp_array` is the DSP count inside the CIM tile array and must be 0 for CIM configurations.)

8. Agree these team rules and write them in `docs/rules.md`:

    - every measured number goes into `runs.csv` with its commit hash;
    - nothing in the report is typed by hand;
    - no LLM ever writes a number into the results.

Output. The repo with a blinking-LED top module that builds to a bitstream with `make bit` and passes a cocotb test with `make sim`.

Review gate.

- ☐ A different member clones it fresh and runs `make sim` and `make bit` successfully.
- ☐ The blinking-LED bitstream runs on the Nexys A7 (and on the ZCU104 as soon as you have it).
- ☐ `runs.csv` exists, with one row from the LED build, including its `board`.

LLM help. Sonnet writes the TCL, the Makefile and the cocotb skeleton. Verify TCL commands against the Vivado TCL reference (UG835). Models invent TCL options.

Step 0.4 — Get and audit the dataset

Week 1–2 · 3 days · Needs: nothing · Lane: can run alongside 0.3

Goal. Find out now whether Sir's dataset can support five honest classes. Don't train anything yet; this step is just questions and counts.

Do.

1. Ask Sir these questions about the dataset when he hands it over, and write the answers in `docs/dataset_audit.md`. The labels alone won't tell you most of this.
    - Classes: which weapons, and is there a non-gunshot class?
    - Collection: how many separate recording sessions (days, locations, shooters)? How was each one recorded?
    - Devices: which microphones or recorders? Was the same shot captured by several devices at once?
    - Format: sample rate, bit depth, channels, file format, and whether it was compressed (MP3 or similar) at any point.
    - Labels: who labelled it, and are the shot times marked?
    - Sources: was any of it taken from the internet, films or games? Those clips need to be kept apart; they can't be treated as field recordings.
    - Use: can results, figures and short audio examples go in the report, the slides and any paper? Can the data leave the lab or go onto Colab or Drive?
    - When: if it isn't ready yet, on what date will it be?
2. Build the pipeline so it doesn't care which dataset it reads. Everything downstream reads `manifest.csv`, never raw folder names. Swapping datasets then means writing one new loader.
3. If Sir's data won't arrive by the end of Week 2, build and test the pipeline on the Kabealo dataset in the meantime (Zenodo DOI 10.5281/zenodo.7004819, CC-BY-4.0, 2148 recordings of four firearms). Stage 2 then isn't blocked; you swap in Sir's data when it arrives.

4. Download UrbanSound8K and ESC-50 (hard negatives for the background class) and Google Speech Commands (fallback workload). Download Kabealo anyway: it is a ready-made cross-dataset test. A model trained on Sir's data and tested on a different dataset's recordings of a matching weapon type is a strong check on whether it learned the gunshot or the recording setup.
5. Write `sw/data/audit.py`. It produces `manifest.csv` with these columns: `path, class, session_id, device, sample_rate, duration, source`. Define `session_id` as the recording session (firing sequence), not the device. If one shot was captured by several devices at once, every one of those recordings shares the same `session_id`; otherwise the same gunshot lands in both train and test. Keep the `device` column too: holding out a device is a second, harder test you can report.
6. Print, per class: the number of clips, the number of distinct sessions, the number of distinct devices, the sample rates and the sources.

Output. `manifest.csv`, plus `docs/dataset_audit.md` containing Sir's answers and the counts table.

Review gate.

- ☐ Every question in item 1 has an answer, or "unknown, asked on DATE."
- ☐ Every firearm class has at least 3 distinct sessions. Otherwise it cannot be split honestly; flag it immediately.
- ☐ The reviewer opens 5 random clips per class and confirms they actually contain a gunshot of that class.
- ☐ Permission to publish results and to use cloud tools is confirmed in writing.

If it fails. If session information doesn't exist, ask Sir how the recordings were made and reconstruct sessions from file dates, folder structure or recording notes. If a class has fewer than 3 sessions, merge classes (for example handgun / long gun / shotgun / background) or ask Sir whether more recordings exist. Decide this with Sir before Stage 2.

LLM help. Sonnet writes the audit script and the loaders. Gemini reads any documentation that comes with the dataset in one go and lists how sessions, devices and labels are recorded. If Sir says the data must not leave the lab, don't paste clips or file listings into any LLM.

STAGE 1 — The complete model in software (Weeks 2–10) ← START HERE

This stage produces the whole model and everything the hardware needs from it: the network definition, the arithmetic spec, trained and quantized weights, a pure-integer reference for every layer, a bit-serial tile model, the session-disjoint accuracy, and frozen per-layer test vectors. No RTL is written in this stage.

What you start with. Step 0.4 delivered `Dataset-Final/` (outside the repo, shared on the team Drive) and `docs/dataset_audit.md`. Five classes, defined by cartridge and gun type:

Class	Clips	Sessions	Sources
`pistol_9mm`	4,145	6	C3GD, Cadre, Kabealo (edge)
`rifle_556_223`	2,268	6	C3GD, edge, Cadre
`revolver_38spl`	866	4	edge, Cadre, C3GD
`shotgun_12ga`	715	5	edge, C3GD
`background`	14,507	3,029	UrbanSound8K, MAD, ESC-50, quiet stretches before edge and Cadre shots

Everything reads clips through `sw/data/manifest.py` (`load_manifest`, `load_audio`), never through folder names. The class index order is fixed by `manifest.classes()`: `pistol_9mm, revolver_38spl, rifle_556_223, shotgun_12ga, background` (0–4). The dense layer's output columns and `argmax` use this order everywhere: Python, test vectors, RTL and the demo GUI.

Two lanes. The model lane (Step 1.1, then 1.6–1.8; owner: whoever owns the model) builds the network definition, the data pipeline and the trained model. The integer lane (Steps 1.3–1.5; owner: whoever will later write RTL) builds the integer machinery on the quick model from Step 1.1. They join at Step 1.10.

Step 1.1 — Network definition and a quickly trained model

Week 3 · 2.5 days · Needs: 0.3, 0.4

Goal. One definition of the network, and realistic weights early, so the integer reference and the tile model can be built and tested on real value ranges while the full training chain runs.

Do.

1. Define the network once, in `sw/models/cnn1d.py`, in TensorFlow/Keras. Every later step imports this file; there must never be two versions of the architecture. It must match `arith_spec.md` §5.1:

   ```
   Input 8000×1 → Conv1D k64 s8 16ch → MaxPool4 → Conv1D k9 32ch → MaxPool4
   → Conv1D k9 32ch → GlobalAvgPool → Dense 32→5
   ```

2. A script prints each layer's output shape, MACs and parameter count. It must reproduce §5.1 exactly: 15,008 weights, 85 biases, 2,731,168 MACs.

3. Cut quick 0.5 s windows around the shots with the simplest available alignment: edge clips from their labelled shot times, C3GD clips from their fixed 0.1 s pre-roll, Cadre clips from the first sample within 30 dB of the peak. Background windows are random 0.5 s crops of background clips. Resample to 16 kHz mono with `load_audio`, and clamp to [−1, 1] (resampling a clipped clip can overshoot full scale).

4. Train for a few epochs on a laptop CPU (Adam, a plain random 80/20 split, class weights) and save `model_quick.keras`.

5. Sanity check only: the model must beat chance and its loss must fall. **Never quote its accuracy.** A random split leaks session information and flatters the number.

What it skips, because those decide the final model: the detector-based windows, grouped splits and cross-validation, the ADC sweep, augmentation, several seeds or architectures, the leakage check and the selection rule. Those are Steps 1.6–1.9.

Output. `cnn1d.py`, the per-layer table, and `model_quick.keras`.

Review gate.

- ☐ The per-layer table matches `arith_spec.md` §5.1 line by line, and is produced by the script.
- ☐ Every later training script imports `cnn1d.py` rather than defining its own copy.
- ☐ The quick model beats chance and its loss fell; its accuracy is not quoted anywhere.

If it fails. If the table disagrees with §5.1, either the model or the spec is wrong. Fix that before anything else is built.

LLM help. Sonnet.

Step 1.2 — Write the arithmetic spec for the whole network

> **Status: done** (`docs/arith_spec.md`, `docs/diagrams/verification_ladder`). Unchanged from v3. Decision D19 (requantize the GAP sum to int8 before the dense layer) is recorded in `docs/decisions.md`. The ASIC rungs of the verification ladder are added in Step 2.5.

Week 1–2 · 5 days · Needs: nothing — the most important step in Stage 1

Goal. A single document that defines every integer operation exactly, so that Python and RTL can be written independently and agree.

Do. Write `docs/arith_spec.md`. It must define:

1. **Number formats.** Activations are signed int8 [−128, 127]. Weights are signed int8. Biases are int32. Accumulators are int32.
2. **Quantization.** Symmetric quantization, per-channel for weights and per-tensor for activations, `x_q = clamp(round(x / s), −128, 127)`, with the rounding mode written out explicitly.
3. **Requantization** from an int32 accumulator to an int8 activation:

   ```
   y = clamp( (acc * M0 + (1 << (n − 1))) >> n , lo, hi )
   ```

   - `M0` is a fixed-width integer (pick 16 bits) and `n` is a shift.
   - State the width of the product `acc * M0`. A 32-bit signed accumulator times a 16-bit multiplier needs up to 48 bits. Write it in the spec as a 48-bit signed intermediate (64-bit in C and Python), so the Python reference, the C code and the RTL cannot disagree about overflow.
   - The `>>` is an arithmetic shift. State what happens for negative values.
   - ReLU is folded in as `lo = 0`.
4. **Bit-serial dot product.** Activation bit-planes are processed `b = 0…7`. Partial sum at bit `b` = the sum of the weights whose activation bit is 1. The accumulator adds `psum << b` for `b < 7` and subtracts `psum << 7` for `b = 7`, because the top bit of a two's-complement number has weight −128.
5. **Layer types.** Convolution as a dot product: for output position `p` and channel `c`, which input samples it reads (start index, stride, padding) and in what order they are split into 16-input chunks. Max pooling on int8 values. Global average pooling as a sum, with the 1/L folded into the next

layer's requantization so no integer division happens in hardware. Dense layers. ReLU (folded into requantization as `lo = 0`) and argmax. Cover every layer type in the starting 1D-CNN now, so the spec doesn't need rewriting later.

6. **Accumulator bound.** Prove that int32 cannot overflow for the largest layer: `max |acc| ≤ inputs × 128 × 128 + |bias|`. Write the number down.
7. **Worked example.** Hand-compute one 4-input dot product bit-serially. Include a −128 activation and a negative weight. Show every cycle.
8. Draw the **verification ladder** as `docs/diagrams/verification_ladder.drawio`: float model → integer reference → Python tile model → RTL simulation → post-synthesis → post-implementation → board, with the pass criterion written on each rung.

Output. `docs/arith_spec.md` and `docs/diagrams/verification_ladder.{drawio,svg}`.

Review gate.

- ☐ Two members independently hand-compute the worked example from the spec, without looking at each other's work. Both get the same answer, and it equals the ordinary `sum(w*a)`.
- ☐ Every layer type in the starting 1D-CNN is defined, including exactly where each window starts and ends.
- ☐ Every operation has its rounding, shift and clamp behaviour spelled out. The reviewer tries to find any operation where two reasonable people could implement it differently. Every one found gets fixed.

If it fails. If the hand computations disagree, the spec is ambiguous. That is exactly what this step exists to catch. Fix the spec, never the arithmetic.

LLM help. Opus helps draft and checks the overflow bound. Then a different model (GPT) independently hand-computes the worked example from the spec. If it disagrees with yours, find out why.

Step 1.3 — Integer reference for the whole network

Week 3–4 · 5 days · Needs: 1.1, 1.2

Goal. `ref_int.py`: a pure-integer forward pass for every layer of the network, implementing `arith_spec.md` exactly. It is the reference every later implementation (Python engine model, RTL, FPGA, ASIC netlists, HLS) must match bit for bit.

Do.

1. `sw/quant/quantize.py`: quantize `model_quick.keras` following the spec. Calibrate the activation scales on real windows. Produce `model_int.npz` with int8 per-channel weights, int32 biases and `(M0, n)` per output channel, including the GAP requantization of D19. The same script quantizes the trained model in Step 1.10.
   - **Make it check its own output.** After producing the artifact, `quantize.py` asserts that every multiplier and shift is within the range `arith_spec.md` allows, and computes the worst-case accumulator for each output channel from the actual weights (sum of absolute weights × 128, plus the bias) and asserts it stays under the int32 bound.
2. `sw/ref/ref_int.py`: a forward pass for every layer type in the spec, using only numpy int32 / int64 arrays. `assert` the dtype at every step. A function `trace(x)` returns every intermediate tensor (`conv1` … `dense` accumulators and outputs, `argmax`).
3. **A second, independent implementation.** A team member who did not write `ref_int.py` writes plain nested-loop versions of `conv1`, `conv2` (with padding) and `dense` from the spec alone. They must agree exactly with `ref_int.py`. These three cover every indexing rule in the network (stride, padding, chunk order, dense layout).

4. **Argmax tie rule.** The logits are int8, so two classes can tie. The spec's rule (§5.9: the lowest index wins) is implemented and tested with a forced tie.

Output. `quantize.py`, `model_int.npz` (quick model), `ref_int.py`, the independent layer versions.

Review gate.

- ☐ No floats anywhere in the forward pass (the dtype asserts pass).
- ☐ The independent versions match `ref_int.py` exactly on 100 random inputs each.
- ☐ The reviewer hand-computes one output of `conv1` and one of `dense`, and both match `trace()`.
- ☐ Every multiplier and shift is within the spec's range, and the worst-case accumulator of every output channel is below the int32 bound (asserted by `quantize.py`).

The quick model's accuracy is never reported. Real accuracy is measured in Step 1.11, with the trained model.

If it fails. Find the first position and channel where two implementations differ and check both against the spec. The usual causes are the window start index, the padding side, the rounding in requantization, and the order of the per-channel scales.

LLM help. Opus for `ref_int.py`. The independent versions must come from a different person and a different model, working only from the spec.

Step 1.4 — Python bit-serial tile model

Week 3–4 · 2 days · Needs: 1.2

Goal. A cycle-level Python model of the hardware tile. It is the bridge between the "math" reference and the RTL, and the tile the Python engine model (Step 2.2) is built from.

Do. `sw/ref/tile_model.py`:

1. Implement the 16×16 tile exactly as the spec says: 8 cycles, with the bit-plane broadcast, the column sums and the MSB subtract.
2. Give it a parameter `BPC` (bits per cycle) with only `BPC = 1` implemented now, the design of `arith_spec.md` §4. `BPC ∈ {2, 4, 8}` (b bit-planes per cycle, b adder trees per column, `8 / b` cycles per dot product) is added in Step 2.5, together with the spec addendum it needs (Decision D24).
3. Test it against numpy `W.T @ a` on 100,000 random vectors, plus edge cases: all activations −128; all activations +127; all weights −128; all zeros.

Output. `tile_model.py` and its tests.

Review gate.

- ☐ 100% match on random and edge cases.
- ☐ The reviewer reads the code against `arith_spec.md` §4, line by line.

LLM help. Sonnet writes it. A different model writes the edge-case list.

Step 1.5 — Test vector generator and file formats

Week 4–5 · 2 days · Needs: 1.3, 1.4

Goal. A fixed, documented format for everything Python hands to RTL, for every layer from the start.

Do. `sw/vectors/emit.py` produces, from any `model_int.npz`:

- `tile_rand.hex`: random tile tests (weights, activations, expected outputs);

- `weights_<layer>.mem`: each layer's weights in `$readmemh` format, laid out by tile slot, with the layout documented (the slot layout is finalized in Step 2.1; until then use one slot per 16-row chunk);
- `requant_<layer>.mem`: the `(M0, n)` table and biases per layer;
- `inputs.hex`: 8,000-sample inputs: real clips, random inputs of mixed amplitude, and 4 extreme vectors (all −128, all 127, all 0, a single spike);
- `expected_<layer>.hex` (`expected_conv1.hex` is the file `arith_spec.md` calls `expected_l1.hex`): the expected output of every layer for each input (accumulators as well as int8 outputs, per `arith_spec.md` §4.6), and `expected_logits.hex` with the class.

Document the byte order, the bit order, the line format and the class index order in `docs/file_formats.md`.

Output. `emit.py`, `docs/file_formats.md`, and vectors for the quick model.

Review gate.

- ☐ The reviewer writes a 10-line reader that parses each file back and recovers the original arrays.

LLM help. Sonnet.

Step 1.6 — Event detector, windows, background and session-disjoint splits

Week 3–4 · 10 days (two people) · Needs: 0.4 · Lane: alongside 1.3–1.5

Goal. Training windows cut exactly the way the hardware will cut them, a background class that cannot be recognised by its source, and splits that share no recording session.

Do.

1. **Event detector.** Write the detector the hardware will use, in Python: energy threshold, refractory period, integer arithmetic only. Save it as `sw/frontend/detector.py`. The RTL detector in Step 4.7 must match it exactly.
2. **Window placement (Decision D22; the window length is D6, Step 1.7).** Decide this before cutting anything. The constraint comes from the data: C3GD clips start about 0.1 s before the shot, and the `c3gd_nj_farm` and `c3gd_ny_gravel_pit` clips are only 0.459 s long. A 0.5 s window that starts 50 ms before the trigger has only about 0.41 s of real audio in those clips. Padding only some clips with zeros makes the padding a session label. Options:
    - **(a) 0.5 s input with 0.40 s of real audio** (0.05 s before the trigger to 0.35 s after it) **and the last 0.10 s zeroed, for every clip from every source and in the live front end.** The 0.459 s clips have about 0.41 s after the window start, so every clip fits with no extra padding; a clip that has less than 0.40 s after its window start is dropped, never padded. The input size and `arith_spec.md` stay unchanged. Recommended.
    - (b) Drop the two 0.459 s C3GD sessions. This costs 9 mm one session and 5.56 and 12 ga their `ny_gravel_pit` session.
    - (c) A 0.4 s input (6,400 samples). This changes `arith_spec.md` §5.1 (`conv1` would give 793 positions), which needs a spec amendment and a fresh review of Step 1.2. The ADC sweep (Step 1.7) can tell whether 0.4 s loses accuracy.
3. **Cut every gun window with the detector,** not with the dataset's timestamps. Use the timestamps (edge labels, the C3GD 0.1 s pre-roll, the Cadre onset) only to check that the detector found the right shots.
4. **Resample to 16 kHz mono, clamp to [−1, 1], and normalize** the same way for every source.
5. **Background.** Random crops of background clips, a new crop every epoch, rejecting windows that are almost silent (ESC-50 pads its clips with zeros). Drop the 55 background clips recorded below 16 kHz, or low-pass every clip to the same band. At least 15% of the background windows in each epoch come from the gun microphones: the `edge-quiet` and `cadre-quiet` stretches, plus UrbanSound8K / ESC-50 events mixed onto those stretches. Also mix gunshots onto UrbanSound8K / ESC-50 noise, so the noise type never identifies the class.

6. **Final test set (Decision D23).** Hold out whole sessions, chosen so every firearm class is tested on sessions from two different datasets. Recommended: **every clip of every class** (background included) whose `session_id` is `c3gd_nj_farm_2` or `edge-day2`, which gives 9 mm 1,143, 5.56 937, .38 251 and 12 ga 216 gun clips plus the `edge-quiet` background cut from the `edge-day2` recordings; then 15% of the *remaining* background sessions, chosen at random by `session_id`. The quiet stretches carry their gun recording's `session_id` and `group`, so selecting by session keeps them on the same side as their shots. The test set is used once, in Step 1.11. Record the choice before any training.
7. **Cross-validation on the rest.** `StratifiedGroupKFold` with `session_id` as the group, for all training and model selection. Report the mean and spread across folds. Note that with the recommended test set, `.38` has only two sessions left for cross-validation (`cadre`, `edge-day1`); report its fold results separately. Save the folds and the test set in `splits.json`.
8. **Device-disjoint split** as a second, harder test: hold out devices within the training sessions.
9. **Random (clip-level) split** as well, to show how much session leakage inflates accuracy.

Output. `detector.py`, the windowing code, `splits.json`, Decisions D22 and D23.

Review gate.

- ▢ A script proves that no `session_id` or `group` appears in more than one fold, or in both the folds and the test set.
- ▢ Every gun window was cut by `detector.py`. The detector finds at least 95% of the known shots, or its miss rate is recorded.
- ▢ No feature of the window (length, padding, sample rate, bandwidth) differs by source: the reviewer checks one window per source, and a script confirms no window was padded beyond the agreed zeroed tail.
- ▢ At least 15% of background windows per epoch come from the gun microphones.
- ▢ The reviewer listens to 10 extracted windows per class and confirms the shot is inside each one.

LLM help. Sonnet for the detector and the windowing. Opus reviews the split for leakage: "how could information from the test set reach training?"

Step 1.7 — ADC sweep

Week 5 · 5 days · Needs: 1.6

Do.

1. Resample to 8 / 16 / 22.05 kHz and requantize to 16 / 12 / 10 / 8 / 6 bits.
2. Train a small fixed CNN at each point, with 3 seeds, on the cross-validation folds.
3. Plot the accuracy heatmap (macro-F1).
4. At the chosen rate and bit depth, compare 0.25 s, 0.4 s and 0.5 s windows. The 1D-CNN's MACs grow in proportion to the window length, and the window is also the wait before an inference can start.

Output. The heatmap and the chosen (rate, bits, window length). If the window differs from 0.5 s, revisit D6 and D22.

Review gate.

- ▢ The chosen point is the cheapest one within 1 percentage point of the best, and that rule was written down before looking at the results.

Step 1.8 — Float models within the hardware budget

Week 6–7 · 15 days (two or three people) · Needs: 1.6, 1.7

Which network. Decided now so nobody is stuck:

- **Hardware model:** a small 1D-CNN on the raw waveform (the network of Step 1.1). It needs no FFT hardware: microphone → engine directly. Its large first layer (a learned filterbank) is also where activation broadcast helps most.
- **Software comparison:** a 2D-CNN on a log-mel spectrogram, plus MFCC + SVM, plus a depthwise-separable CNN. These answer the research question "waveform vs spectrogram" in accuracy and MACs.
- **Promotion rule** (agree it now, before training): the 2D-CNN only goes into hardware if it beats the 1D-CNN by more than an agreed margin (suggestion: 5 points of macro-F1) on cross-validation. Even then it gets synthesized and simulated, not demoed live.

Budget, for every model: ≤ 4 M MACs per inference. Parameters are limited by where the CIM weights live, which on the FPGA is distributed RAM (LUTRAM): about 1.2 Mbit on the XC7A100T (check the Artix-7 datasheet, DS180), shared with logic. Provisional limit: 30 k parameters (240 kbit at 8 bits), confirmed in Step 2.1. The limit is set by the Nexys, the smaller board, so the same model runs everywhere. On the ASIC, weight storage is area: about 120 kbit for the starting network, a modest flip-flop area at 45 nm. Layers are limited to conv, maxpool, ReLU, global-average-pool and dense. Batchnorm is allowed during training but gets folded in at export.

Tile-friendly shapes. Each layer's output channels a multiple of 16 (the tile width) and each layer's `k × Cin` a multiple of 16 (the tile height). The starting network's conv layers already are; the dense layer (5 outputs) uses 5 of a tile's 16 columns, with the other 11 holding zero weights (`arith_spec.md` §5.7).

More classes later. Up to 16 classes cost nothing in hardware (one tile pass for `dense`). If the 6+1 side experiment (adding .22 LR and 7.62×39 from `Dataset/`) is run, it reuses this network with a 32→7 dense layer.

Do.

1. Train each model on every cross-validation fold, with augmentation (gain, noise, time shift, reverberation, the mixing of Step 1.6), class weights and per-epoch background sampling.
2. Log the fold-mean and spread of macro-F1, balanced accuracy and per-class recall; the random-split numbers; MACs and parameters. Report accuracy per source as well as per class.
3. **Leakage check:** train a small model on the four firearm classes using only the 50 ms before each trigger, which contains background noise and no gunshot. Chance is 25%. If it scores well above chance, the classes can be told apart by recording conditions alone, and the main accuracy is inflated by that amount.

Output. Model files, the accuracy-vs-MACs Pareto plot and a comparison table.

Review gate.

- ☐ Every model's MACs are computed by a script, not estimated.
- ☐ Cross-validated macro-F1 (mean and spread), per-class recall and random-split accuracy are all reported.
- ☐ The leakage check scored close to chance, or its result is reported next to the main accuracy.
- ☐ The test set has not been touched. All model selection used the folds only.

If it fails. If cross-validated accuracy is far below what Sir expects, decide with him by Week 8: merge classes (handgun / rifle / shotgun / background), add data from `Dataset/` (Cadre's other guns, C3GD's other calibers), or widen `conv3` to 64 channels (about 3.3 M MACs and 25 k weights, still within budget). Don't carry the problem forward.

LLM help. Sonnet for the training code. Opus to review the protocol for leakage.

Step 1.9 — Model selection gate

Week 8 · meeting, 1 day · Needs: 1.8

The team decides the hardware model using the promotion rule from Step 1.8, and records it in `docs/decisions.md`. **From this point the architecture is frozen**: the layer shapes in `cnn1d.py`, `arith_spec.md` §5 and the RTL parameters are the same thing. A later change re-opens Steps 1.2, 1.3, 1.5 and every downstream step that has started.

Step 1.10 — Quantize the selected model

Week 8–9 · 8 days · Needs: 1.9, 1.3

TensorFlow note. Train in Keras, but do not use the TFLite converter's int8 output as your hardware reference. TFLite quantizes activations asymmetrically with a zero point, and `arith_spec.md` defines symmetric activations with no zero point, which is much cheaper in hardware. Run your own PTQ in numpy (`quantize.py` from Step 1.3) from the Keras weights and calibration activations, exactly as the spec says. If PTQ loses too much accuracy, do QAT with `tf.quantization.fake_quant_with_min_max_vars` using symmetric ranges. TFLite is still useful as an accuracy cross-check.

Do.

1. PTQ: per-channel weights, per-tensor activations, percentile calibration on training-fold windows only.
2. QAT only if PTQ loses more than 1 point of macro-F1.
3. Bit-width sweep: W8A8, W6A6, W4A8, W4A4 (reported; the hardware stays W8A8).
4. Fold batchnorm into the conv weights and check numerically that folded = unfolded.

Review gate.

- ☐ W8A8 integer macro-F1 is within 1 point of float on cross-validation.
- ☐ Batchnorm folding is verified numerically.
- ☐ `quantize.py`'s range and accumulator-bound assertions pass on the trained weights.

LLM help. Opus. The scaling math is where silent errors live.

Step 1.11 — Integer reference on the trained model, and the final test

Week 9–10 · 3 days · Needs: 1.10, 1.3

Do.

1. Export the trained, quantized model as `model_int.npz` (same format as Step 1.3).
2. Run the cross-validation windows through `ref_int.py` and confirm its accuracy equals the quantized model's.
3. **Run the held-out test set once,** through `ref_int.py`. This is the project's accuracy, because every hardware implementation is required to be bit-exact with `ref_int.py`, so the hardware's accuracy is the same number by construction. Report: macro-F1, balanced accuracy, per-class recall, the confusion matrix, per-source results, and alongside them the device-disjoint and random-split numbers.
4. If the selected model uses anything the spec doesn't cover (for example the 2D-CNN's 2D windows), add it to `arith_spec.md` and `ref_int.py` first, and have two members hand-check one output of the new layer type.

Review gate.

- ☐ `ref_int` accuracy equals the quantized-model accuracy on cross-validation.
- ☐ The test-set run happened once, and its commit hash is in `runs.csv` (`stage = 1.11`, `board = sw`).

- [ ] Any new layer type was added to the spec and hand-checked by two members.

Step 1.12 — Freeze the model and its test vectors

Week 10 · 2 days · Needs: 1.11, 1.5, 2.1 (slot layout)

Goal. One tagged version of the model that every hardware step builds against.

Do.

1. Run `emit.py` on the trained `model_int.npz`: weights and requantization tables for every layer, 200 test clips (real windows from the test set, spread over classes and sources) plus the random and extreme inputs, and the expected tensors of every layer.
2. Tag the repository `model-v1`. Record the tag, the commit and the hash of every vector file in `docs/file_formats.md`.
3. Prepare the Milestone A demo: the per-layer table, the cross-validation and test results, a live `ref_int` run on a few clips.

Review gate.

- [ ] The round-trip parser from Step 1.5 works on every new file.
- [ ] `expected_logits.hex` equals `ref_int.py` on all 200 clips, rerun by the reviewer from the tag.

> **Milestone A — "The complete model works, in integers."** The architecture, weights, quantization and integer reference are frozen, the session-disjoint accuracy is known, and per-layer test vectors exist for every layer. Demo it to Sir. Line it up with the mid-term evaluation (Step 0.1 question 7), together with the ASIC flow bring-up (Step 5.1) and the SRAM-CIM bitcell schematic (Step 5.2).

STAGE 2 — Engine architecture on paper and in Python (Weeks 7–11)

Lane. Steps 2.1 and 2.3 can be drafted from the starting network in Week 7, because its shape is already fixed in `arith_spec.md` §5.1; they are finalized after the selection gate (Step 1.9). Step 2.2 needs the frozen model.

By the end of this stage the whole engine exists as documents and as a bit-exact Python model. RTL in Stage 3 is a translation, checked block by block against that model.

Step 2.1 — Mapping spec: how every layer runs on tiles

Week 7–9 · 8 days · Needs: 1.2; finalized after 1.9

Goal. On paper first: exactly how each layer of the frozen model is scheduled on `NUM_TILES` tiles.

Do. Write `docs/mapping_spec.md`. It must cover:

1. **Conv1D as dot products.** Each output (position `p`, channel `c`) is a dot product over `k × Cin` inputs, split into chunks of 16 rows, with partial sums accumulated across chunks (`arith_spec.md` §5.3). Requantization happens only after the last chunk. The dense layer is the same with `L = 1`.
2. **Weight placement.** Which weights go in which tile and which slot, for every layer. Slots needed = Σ over layers of `ceil(K / 16) × ceil(Cout / C)` (for the starting network with `C = 16`: 4 + 18 + 36 + 2 = 60; the dense layer uses 5 of a slot's 16 columns). Decide whether all weights stay resident (weight-stationary for the whole network, no reloads during inference) or tiles are reloaded between layers, and count the weight writes per inference either way. Check capacity: LUTRAM of the Nexys A7-100T (DS180), and storage bits for the ASIC.
3. **Parallelism, `n_macs`.** Define it here and use it everywhere: `n_macs` = MACs completed per cycle = `16 × C × NUM_TILES × BPC / 8`, with 16 rows per tile (fixed by `arith_spec.md` §5.3), `C` columns per tile, and `BPC` bit-planes per cycle. One 16 × 16 tile at `BPC = 1` gives 32. The buildable sweep values are 8 (`C = 4`), 16 (`C = 8`), then 32, 64, 128, 256, 512, 1024 (`C = 16`, `NUM_TILES` = 1 … 32), and `BPC` multiplies them on the ASIC. The baseline gets the same `n_macs`.
4. **Activation buffers.** Ping-pong between layers and their sizes (`conv1` output 993 × 16 int8 is the largest). Check that they fit BRAM on the Nexys.
5. **The schedule.** The cycle count per layer as a formula in `C`, `NUM_TILES`, `BPC` and the layer shape, and the total for the frozen model.
6. **Instrumentation.** Counters for cycles, weight writes (0 during inference if weights are resident), weight reads and activation reads.
7. **Requantization units.** How many there are and how many output columns share each (a parameter; start with one per tile). Show in the schedule that the tile array never stalls waiting for one.
8. **Diagrams.** `docs/diagrams/layer_mapping.drawio`: each conv layer split into 16-row chunks, which tile and slot holds which weights, and the ping-pong buffers. Also draft `docs/diagrams/system.drawio` (microphone → I²S receiver → event detector → ring buffer → engine → host link), which replaces synopsis Figure 5.1 and is finalized in Step 4.8.

Review gate.

- ☐ The reviewer computes the total cycles for the frozen model from the formula. Write the number down; Steps 2.2 and 3.3 must reproduce it.
- ☐ LUTRAM and BRAM use fit the Nexys with a margin of at least 20%.
- ☐ The layer-mapping diagram is checked against the written schedule by a non-author.
- ☐ The number of requantization units and their sharing are stated, and the schedule shows no stall waiting for them.

LLM help. Opus. This is a judgment-heavy architecture step.

Step 2.2 — Python engine model: the golden model of the complete engine

Week 10 · 5 days · Needs: 2.1, 1.4, 1.12, 2.5

Goal. A block-level Python model of the complete CIM engine that follows `mapping_spec.md` exactly: the same blocks, buffers, loops and counters the RTL will have. It is bit-exact with `ref_int.py` and gives the reference cycle count. Every RTL block in Stage 3 is tested against its counterpart here.

Do. `sw/engine_model/`:

1. One Python class per RTL block of Step 3.2 (`act_buffer`, `im2col_addr`, `requant_pc`, `maxpool`, `gap_sum`, `argmax`, `layer_seq`, `perf_counters`), built around `tile_model.py`, with the same interfaces the RTL will have.
2. A top level that runs the frozen model on any input, cycle-counted, with parameters `NUM_TILES` and `BPC`.
3. A dump mode that writes every block's outputs per layer, in the formats of `docs/file_formats.md`, so a failing RTL block can be compared with its Python counterpart directly.

Output. `sw/engine_model/` and its tests.

Review gate.

- ☐ Bit-exact with `ref_int.py` on all 200 clips and the extreme vectors, every layer, for `NUM_TILES ∈ {1, 2, 4}` and `BPC ∈ {1, 2, 4, 8}`.
- ☐ The cycle count equals the formula of Step 2.1 for every configuration.
- ☐ The weight-write counter is 0 during inference (if weights are resident).

LLM help. Sonnet writes it from `mapping_spec.md`. Opus reviews it against the spec.

Step 2.3 — Baseline engine spec and fairness review

Week 8–10 · 8 days · Needs: 2.1 (draft); finalized with it

Goal. Define the conventional ("von-Neumann") baseline before building it, and get it reviewed for fairness while changes are still cheap.

Do. Write `docs/baseline_spec.md`. The baseline must have:

- the same MAC count, the same adder trees, the same precision, the same requantization, the same clock and the same model;
- **one difference:** weights live in a memory away from the MACs, and a controller fetches them into a small buffer before use. The MACs stall when they are starved;
- a prefetch / double-buffer, so it is a competent baseline and not a crippled one;
- on the FPGA, a default weight bandwidth equal to the most the weight BRAMs can deliver, not one narrow port. Weights filling N BRAM36 blocks can be striped across all of them and read at full width (two 36-bit ports in true dual-port mode, or one 72-bit port in simple dual-port mode; check UG473), which gives roughly N × 72 bits per cycle. A single 64-bit port is the first thing an examiner will call a strawman;
- `WPORT` swept from narrow up to that maximum, so the report shows exactly where the baseline stops being limited by bandwidth;
- **on the ASIC,** the baseline's weight memory uses the same storage cells as the CIM tile it is compared with (flip-flops against the flip-flop tile, latches against the latch tile), with a read port and the same fetch controller. The storage is then identical and only the dataflow differs.

Also write into the spec, before any results exist: at full bandwidth, the baseline may match CIM's latency at this model size. If it does, the comparison moves to cost (LUTs, FFs or μm² per MAC-per-cycle, and energy per inference on the ASIC) and to how each design scales. That is still a valid result.

Diagram. `docs/diagrams/cim_vs_baseline.drawio`: both engines side by side, same MAC array, same adder trees, same requantization, with the only difference being where weights come from and the stall path that creates. Draw it as part of writing the spec. This is the figure your viva turns on.

Review gate (fairness).

- ▢ A team member who did not write the spec answers in writing: "If I were the examiner, how would I argue this baseline was made deliberately weak?" Every point is either fixed or answered in the spec.
- ▢ Opus and GPT are each given the spec with the same question. Their points are addressed.
- ▢ The spec answers "why not just keep the weights in registers next to the MACs?" That is what a fully partitioned HLS design does (Stage 6), so the HLS result becomes a third reference point.
- ▢ The side-by-side diagram exists, and a non-author confirms the two datapaths differ only in weight supply.
- ▢ Sir has seen it.

Step 2.4 — Cycle model for both engines

Week 10–11 · 4 days · Needs: 2.2, 2.3

Goal. Predict the cycle counts of both engines before any RTL exists.

Do.

1. `sw/simcycle/`: the CIM side is the cycle count of the Python engine model (Step 2.2); add the baseline's fetch and stall model from the spec, parameterized by `n_macs`, `WPORT` and `BPC`.
2. Sweep `n_macs ∈ {8, 16, 32, 64, 128, 256, 512, 1024}` (as defined in `mapping_spec.md`) and `BPC ∈ {1, 2, 4, 8}`.

Output. A predictions table and plot, saved as `results/predictions_v1.csv`.

Review gate.

- ▢ The CIM predictions equal the Python engine model's cycle counts.
- ▢ Prediction files are labelled "prediction." None of these numbers goes into any claim. They exist only to be checked against simulation and hardware in Stage 7.

LLM help. Sonnet writes it. Opus checks the baseline's stall model.

Step 2.5 — Rules for target-neutral RTL, and the BPC addendum

Week 8 · 2 days · Needs: 1.2, 1.4, 5.1 (tool list)

Goal. One set of RTL rules so the same `rtl/core/` synthesizes in Vivado and in Genus without edits.

Do.

1. Write `docs/rtl_rules.md`:
   - no vendor primitives, no `(* use_dsp *)` or `(* ram_style *)` attributes and no `initial` blocks in `rtl/core/`. FPGA attributes go in the board wrappers or behind `` `ifdef FPGA ``;
   - weights and requantization tables are loaded only through write ports: by the testbench in simulation (the testbench may use `$readmemh`), by UART commands on the boards (Step 4.1), and by the testbench on ASIC netlists. No `$readmemh` and no `initial` block in `rtl/core/`. The weight-write counter counts these writes only outside inference;

    - one reset style (synchronous, active-low) and one clock domain in the core; clock-domain crossings only in the front end, from proven templates;
    - weight storage only through `rtl/core/weight_store.sv`, with `STORAGE ∈ {"lutram", "ff", "latch", "macro"}`. On the FPGA `"lutram"` infers distributed RAM; on the ASIC `"ff"` and `"latch"` give standard-cell storage, and `"macro"` instantiates the SRAM-CIM macro's Verilog model (Step 5.3);
    - parameters for `NUM_TILES`, `BPC`, `n_macs` and `WPORT` (baseline); no hard-coded layer sizes outside the layer table;
    - the lint command that must pass with both `FPGA` and `ASIC` defined.
2. Write the **BPC addendum** to `arith_spec.md` §4 as a new subsection (§4.7): with `BPC = b`, bit-planes `jb … jb+b−1` are summed in the same cycle, each shifted by its weight, with the MSB plane still subtracted. The result is identical to `BPC = 1`; only the cycle count changes. Record it as Decision D24. Step 1.2's text is not edited.
3. Extend `tile_model.py` (Step 1.4) to `BPC ∈ {2, 4, 8}` from the addendum and rerun its tests for every `BPC`.
4. Add the ASIC rungs to the verification ladder (only `docs/diagrams/verification_ladder.png` exists; redraw it as `verification_ladder.drawio` first, then export SVG): RTL → Genus netlist (Conformal LEC + gate-level simulation) → post-layout netlist with SDF (gate-level simulation), each with its pass criterion "bit-exact on the stated vectors".

Review gate.

- ☐ A reviewer finds no vendor primitive or FPGA attribute in `rtl/core/` (grep in the gate script).
- ☐ Two members hand-compute one dot product with `BPC = 2` from the addendum and get the `BPC = 1` answer.
- ☐ `tile_model.py` matches numpy on 100,000 random vectors and the edge cases for every `BPC ∈ {1, 2, 4, 8}`.

LLM help. Opus drafts the rules; verify any tool-specific statement against the Vivado and Genus documentation.

STAGE 3 — RTL of the complete engine, in simulation (Weeks 10–16)

Every RTL block is written for the frozen model and tested against its Python counterpart from Step 2.2 the day it is finished. The complete engine is then integrated in simulation, one layer at a time. It goes to the board (Stage 4) and to the ASIC flow (Stage 5) only once it is bit-exact in simulation on all 200 clips.

Random vectors still have a job. The tile and block unit tests keep using random and extreme values (the all −128 case, the single spike, random weights), because real audio never reaches the corners where bit-serial bugs live.

Step 3.1 — CIM tile RTL and its testbench

Week 10–11 · 10 days · Needs: 1.4, 1.5, 2.5

Goal. A synthesizable, verified `cim_tile`, the core of the whole project, that synthesizes for both the FPGA and the ASIC.

Do.

1. **Diagrams first,** from `arith_spec.md` only: `docs/diagrams/tile_datapath.drawio` and the bit-serial timing diagram (WaveDrom). Put the bit width on every wire, mark every pipeline register, show the MSB subtract explicitly, and show the `BPC` adder trees. Redraw the clean version once the testbench passes.
2. Write `rtl/core/cim_tile.sv`:
   - parameters `R=16, C=16, WBITS=8, ABITS=8, DEPTH, BPC, STORAGE`;
   - weights in `weight_store` (per row, `DEPTH × (C·WBITS)`), with a write port for preload and a read port addressed by slot;
   - per cycle, the inputs are `BPC` bit-planes of `a_bits[R]` and a bit index, and it performs the AND mask, the adder tree(s) per column and the shift-accumulate with the MSB subtract;
   - requantization is not part of the tile. It lives in `rtl/core/requant_pc.sv`, outside the macro, and may use DSP slices on the FPGA.
3. Write `rtl/core/adder_tree.sv`, parameterized, with an optional pipeline register stage.
4. Write the cocotb testbench `tb/test_cim_tile.py`. It reads `tile_rand.hex` and runs, for every `BPC ∈ {1, 2, 4, 8}` and `STORAGE ∈ {"lutram", "ff", "latch"}`:
   - 10,000 random tests;
   - directed tests for every edge case from Step 1.4;
   - a preload/reload test (write slot 0, compute, write slot 1, compute, return to slot 0 → still correct).
5. Lint: `verilator --lint-only -Wall`, with `FPGA` and with `ASIC` defined.

Output. The RTL, the testbench, the lint logs, and the datapath and timing diagrams.

Review gate.

- ☐ 100% pass, random and directed, for every `BPC` and `STORAGE`.
- ☐ The lint log is clean: no unintended latches (the `"latch"` storage is the only intended one), no width warnings left unexplained.
- ☐ The reviewer deliberately breaks the MSB subtract (changes it to an add) and confirms the testbench fails.
- ☐ A reviewer who did not draw it checks the datapath diagram against the RTL: every block is a module, every module is a block, and every bit width matches.

If it fails. Compare cycle by cycle against `tile_model.py`. Add a debug dump of `psum` at each bit position from both.

LLM help. Sonnet writes the RTL and the testbench, but in separate conversations, each from `arith_spec.md` only. If a bug survives an hour, take it to Opus with the spec, the RTL and the failing vector.

Step 3.2 — Engine blocks, each tested alone

Week 11–13 · 15 days · Needs: 3.1, 2.2

Goal. Every block the complete engine needs, each bit-exact with its Python counterpart before it is connected to anything.

Diagram first. `docs/diagrams/engine.drawio` (tiles, buffers, window addressing for every layer, requantization, pooling, GAP, dense, argmax, data widths) and the controller state machine with the cycles per state. The state machine is where the cycle-count formula comes from, and it must agree with Step 2.1.

Do. Build each block for the full network and test it alone against `sw/engine_model/`, the day it is finished:

Block	What it does
`act_buffer`	Holds the 8,000-sample input and each layer's output; ping-pong between layers
`im2col_addr`	For every conv layer: the input addresses of output position `p`, chunk by chunk, with stride and padding (`arith_spec.md` §5.4); the dense layer as `L = 1`
`requant_pc`	Per-channel requantization and ReLU (outside the macro; the expected DSP user on the FPGA)
`maxpool`	int8 max over 4 positions
`gap_sum`	Sum over 62 positions, then requantization (Decision D19)
`argmax`	Picks the class; lowest index wins a tie
`layer_seq`	Walks the layer table from the mapping spec
multi-tile control	Spreads one layer's work across `NUM_TILES` tiles
`perf_counters`	Cycles, weight writes, weight reads, activation reads

For `requant_pc`, draw the multiplier, shift and bias at random across their full legal ranges as well as the accumulator, so no model can hand the hardware a combination the test hasn't seen. For `im2col_addr`, test the first and last position of every layer and every padded position.

Output. The blocks and their unit tests, and the diagrams.

Review gate (per block).

- ☐ The unit test passes and is bit-exact with the Python block.
- ☐ Lint is clean.
- ☐ The reviewer breaks one thing in the block and confirms the test catches it.

LLM help. Sonnet for each block and its test. Opus for any "mostly matches" bug: 99% agreement nearly always means a rounding, sign or off-by-one error.

Step 3.3 — Integrate the complete engine in simulation, one layer at a time

Week 13–14 · 10 days · Needs: 3.2, 1.12

Do. Assemble the engine with the frozen weights, one piece at a time, in this order, and make each addition bit-exact in simulation before adding the next:

`conv1` → `pool1` → `conv2` → `pool2` → `conv3` → `gap` → `dense` → `argmax` → several tiles (`NUM_TILES > 1`)

Each addition compares every intermediate tensor with `expected_<layer>.hex` and with the Python engine model's dump, and logs a row in `runs.csv` (`board = sim`). Finish with all 200 clips and the extreme vectors.

Output. The complete CIM engine RTL (`rtl/core/cim_engine.sv`) and its testbench.

Review gate.

- ☐ Every addition passed before the next one was added (each has a row in `runs.csv`).
- ☐ Bit-exact on every layer for all 200 clips and the extreme vectors.
- ☐ The cycle count equals the formula of Step 2.1 and the Python engine model.
- ☐ The weight-write counter reads 0 during inference (if weights are resident).
- ☐ A non-author checks the engine block diagram and state machine against the RTL.

If it fails. The bug is almost always in the last addition, which is the point of adding one at a time. The usual causes: the order of partial-sum accumulation across chunks; per-channel requantization indexing; off-by-one errors in `im2col` at layer edges (padding, stride); the window start (`8p`, not `8p + 1`) and the last `conv1` position (index 992).

Step 3.4 — Baseline engine RTL

Week 13–15 · 12 days · Needs: 2.3, 3.2 · Lane: alongside 3.3

Do. Reuse the MAC array and all the Step 3.2 blocks. Replace weight storage with the weight memory and fetch controller from the spec: BRAM on the FPGA, the same `weight_store` cells behind a read port on the ASIC.

Review gate.

- ☐ Bit-exact on all 200 clips.
- ☐ The baseline and CIM produce identical outputs on all clips: the same arithmetic with different data movement. If they differ, one of them is wrong.
- ☐ The cycle count matches the cycle-model prediction within 10%. If not, explain the difference before moving on.

Step 3.5 — Parameter sweep in simulation

Week 15–16 · 4 days · Needs: 3.3, 3.4 · Lane: alongside 4.5

Do. Run both engines at each `n_macs` and `BPC` value in simulation and log the cycles. Compare against `predictions_v1.csv`.

Review gate.

- ☐ Every configuration is still bit-exact.
- ☐ A table of predicted vs simulated cycles exists, with every difference explained.

STAGE 4 — FPGA implementation, boards and demo (Weeks 5–18)

Step 4.1 is an early lane: it builds and tests the path between the PC and the board long before there is an engine to put on it. Everything else in this stage starts once the complete engine passes simulation (Step 3.3).

Step 4.1 — Board bring-up and the UART link

Week 5–8 · 5 days · Needs: 0.3 · Lane: alongside Stage 1

Goal. A tested path between the PC and the Nexys A7, so the complete engine goes onto the board the day it passes simulation.

Do this in strict order. Each part works before the next starts.

1. Blinking LED on the Nexys A7 (from Step 0.3).
2. **UART loopback.** The board echoes bytes. A host script sends 1,000 random bytes and checks the echo.
3. **UART protocol.** Define and document it in `docs/uart_protocol.md`: a start byte, a command byte, a length, the payload and a checksum. Commands: write weights (tile, slot, data), read back weights (tile, slot), write and read back the requantization tables and biases, load an input (8,000 bytes), start the engine, read back the class, the five logits and the cycle count (uint32), read the performance counters, and (for debugging) read back one layer's output buffer. At 115,200 baud an input takes about 0.7 s; use a higher baud rate if both ends support it.
4. **A stand-in engine.** Put a trivial block behind the protocol that has the engine's interface: it accepts an input, waits a programmable number of cycles, and returns a known function of the input as "logits" and the cycle count. `host/run_engine.py` exercises every command against it.

Output. The UART RTL, `docs/uart_protocol.md`, `host/run_engine.py`, and the results row in `runs.csv`.

Review gate.

- ☐ The loopback test passes with 1,000 random bytes.
- ☐ Every protocol command works against the stand-in engine, including a checksum error that is detected.
- ☐ A non-author has reviewed the protocol document.
- ☐ The reviewer reproduces the stand-in test on their own laptop.

LLM help. Sonnet for the UART RTL and host scripts. Write the UART receiver from a known-good reference design rather than letting a model invent one.

Step 4.2 — Synthesize the complete engine

Week 14–15 · 3 days · Needs: 3.3, 0.3

Do.

1. Run `make synth TOP=cim_engine BOARD=nexys` at 100 MHz.
2. Read the hierarchical utilization report (LUT, FF, LUTRAM, BRAM, DSP per module; `report_utilization -hierarchical`, confirm the option in UG835) and the timing report (WNS). The per-module view is how you separate the tile array from requantization.
3. Run a post-synthesis functional simulation on 5 clips.

Review gate.

- [ ] DSP = 0 inside the tile array in the hierarchical report. Any DSPs in the design belong to requantization and are reported as such, with the whole-design total.
- [ ] WNS ≥ 0 at 100 MHz, or you have recorded the achieved frequency and the critical path.
- [ ] The post-synthesis simulation is bit-exact.
- [ ] A row is added to `runs.csv`.

If it fails. If WNS < 0, add an adder-tree pipeline stage (a parameter change). If DSPs appear inside the tile array, check that the FPGA wrapper sets the attribute and that `rtl/core/` follows `docs/rtl_rules.md`. DSPs in the requantization unit are expected. If LUTRAM runs out, recheck the capacity table of Step 2.1.

LLM help. Gemini reads the full synthesis and timing reports and identifies the critical path and any inferred DSP.

Step 4.3 — The complete engine on the Nexys A7

Week 15 · 4 days · Needs: 4.1, 4.2

Goal. The first hardware result: the whole network, bit-exact, on a board.

Do.

1. Replace the stand-in engine of Step 4.1 with the complete CIM engine.
2. `host/run_engine.py` loads the frozen `model-v1` weights and requantization tables over UART, reads them back to check them, then sends all 200 clips and the extreme vectors, reads back the class, the logits and the cycle count, and compares them with `ref_int.py`. For any mismatch, it reads back the layer buffers and finds the first layer that differs.

Output. The results rows in `runs.csv`.

Review gate.

- [ ] The weights read back from the board equal `weights_<layer>.mem`, and the weight-write counter does not change during inference.
- [ ] Class and logits match the reference for every input, on the Nexys A7.
- [ ] The board cycle count equals the simulation cycle count.
- [ ] The reviewer reproduces the run on their own laptop.

> **Milestone B — "The complete model runs bit-exact on the board."** Demo it to Sir.

LLM help. Sonnet for the host script.

Step 4.4 — The complete engine on the ZCU104

Week 15–16 · 4 days · Needs: 4.3, and the ZCU104 in hand

Goal. The same engine on the final FPGA platform.

Do.

1. Create `rtl/board/zcu104/` and the ZCU104 constraints. Take pin names and locations from AMD's master constraints file for the board, never from an LLM. A wrong pin assignment can look fine and drive the wrong thing.
2. Get the blinking-LED build running on the ZCU104 first.
3. Choose how the PC talks to the fabric. Try them in this order:

- **Option A, fabric only.** Keep the same UART block and protocol as the Nexys, with an external 3.3 V USB-UART module on a Pmod connector. The design is programmed over JTAG. Test on day one that this works on your board setup, because the Zynq's processor system normally has to be considered even for fabric-only designs. The clock comes from the board's fabric clock; check the user guide for the pin and frequency.
- **Option B, through the processor system.** A Zynq UltraScale+ block design: the fabric clock from the processor system, a small AXI-Lite register block (start, status, cycle counter, weight-write counter, input and output buffers), and a host program on the board's own USB-UART, either bare-metal in Vitis (C) or Python under PYNQ.

Use Option A if it works. Record the choice as Decision D15.

4. Run the 200 clips through the engine on the ZCU104.

Review gate.

- ☐ Every output matches `ref_int.py`, the same result as Step 4.3.
- ☐ The cycle count equals the Nexys cycle count exactly. Cycles come from the design, not the board; a difference means a wrapper bug.
- ☐ `make bit BOARD=zcu104` builds from a clean clone, and its row is in `runs.csv` with `board = zcu104`.
- ☐ Decisions D14 (measurement platform) and D15 (host interface) are recorded.

If it fails. If the ZCU104 isn't available, mark this step Cut in the tracker. Everything works on the Nexys alone, with the ZCU104 results as an extension.

LLM help. Sonnet writes the wrapper and TCL; verify block-design and constraint details against the ZCU104 user guide (UG1267) and the Vivado documentation. Opus if the PS–PL clocking or reset behaves strangely.

Step 4.5 — Build all configurations

Week 15–16 · 8 days · Needs: 3.3, 3.4, 4.2

Do.

1. `make synth` (and `make bit` for the configurations that will be measured) for CIM and baseline × each `n_macs`, plus the HLS variant (Stage 6) when it is ready, once per board. On the Nexys, sweep as far as it fits. On the ZCU104, extend the sweep to larger MAC counts (for example 256 and 512) if they fit and close timing. Use the same clock target for CIM and baseline on a given board. `BPC` stays 1 on the FPGA unless Step 3.5 shows a reason to sweep it.
2. Harvest the reports into `runs.csv` automatically.

Review gate.

- ☐ Every configuration has WNS ≥ 0 at its target clock, or a recorded achieved frequency.
- ☐ Every CIM configuration shows DSP = 0 inside the tile array. Requantization DSPs are reported separately, and both `dsp` and `dsp_array` are in `runs.csv`.
- ☐ A post-route timing simulation on 20 clips is bit-exact for at least the largest CIM and baseline configurations.

If it fails. If large configurations don't fit or don't close timing, drop the largest `n_macs` point and report it as "did not fit the part." That is a valid observation, not a failure, and the ASIC sweep (Step 5.7) continues past it.

LLM help. Gemini on the full Vivado logs. Sonnet on the harvesting script.

Step 4.6 — Full test set on the boards

Week 16 · 4 days · Needs: 4.5, 4.3, 4.4 (D14 picks the board)

Do. Run every test-set window through the CIM engine and the baseline on the measurement board, logging the class, the logits and the cycle count.

Review gate.

- ☐ 100% logit match with `ref_int.py` on the full test set, for both engines.
- ☐ Board cycle counts equal the simulation cycle counts.

Step 4.7 — Microphone path

Week 12–15 · 12 days · Needs: 0.1 (mic bought), 1.6 (detector) · Lane: alongside Stage 3

Which microphone. The Nexys A7's onboard PDM microphone needs a PDM-to-PCM filter chain that doesn't carry over to the ZCU104, which has no microphone. Use an external I²S MEMS microphone on a Pmod: the same receiver works on both boards. Use the onboard PDM microphone only as a quick sanity check if the I²S microphone hasn't arrived. Record the choice as Decision D16.

Do this in strict order.

1. **I²S receiver.** Mic → FPGA → raw samples streamed over UART → host plots the waveform. Check it visually (clap, speech).
2. Truncation to the bit depth and rate chosen in Step 1.7.
3. **Event detector,** matching `sw/frontend/detector.py` bit for bit: energy threshold, refractory period, a ring buffer that captures the window exactly as Step 1.6 placed it (including the zeroed tail if D22 chose option a).
4. **Clock-domain crossing** from the I²S clock to the system clock, using a known-good asynchronous FIFO or synchronizer template.

Review gate.

- ☐ Recorded waveforms match a reference recording of the same sound made on a laptop.
- ☐ The RTL detector and `detector.py` trigger on the same samples for 100 recorded events.
- ☐ 1,000 synthetic clicks over 10 minutes are all detected with no lockups.
- ☐ The microphone path works on the board used for the final demo.

LLM help. Sonnet for the host plotter. Do not let a model write the CDC logic from scratch; use a proven template and have Opus review it.

Step 4.8 — Live demo integration

Week 17–18 · 8 days · Needs: 4.4 or 4.3, 4.7

Do. Connect the microphone path to the engine. `host/demo_gui.py` shows the waveform, detected events, the predicted class, a confidence (softmax computed on the host from the logits) and the measured cycles. Record about 10 minutes of ordinary room and street sound on the demo microphone and check the false-alarm rate on it.

Then finalize `docs/diagrams/system.drawio` so it matches what you actually built: every block, the clock-domain crossing marked, and both board wrappers. It replaces synopsis Figure 5.1 and is the figure you present at the defence.

Review gate.

- ☐ Recorded test gunshots, played through a speaker, are classified on screen.
- ☐ The demo runs for 30 minutes without a reset.

- [ ] The false-alarm rate on the 10-minute background recording is reported.
- [ ] The system diagram matches the built system, checked by a non-author, and is exported as SVG or PDF.

Speaker playback changes the sound, so live accuracy will be lower than the test-set accuracy. Report it separately as "live demo accuracy" and never mix it with the test-set number.

> **Milestone C — "It works live."**

STAGE 5 — ASIC implementation (Weeks 3–19)

What this stage delivers (Milestone D). The same CIM engine at a process node, compared with the baseline under identical conditions, in three forms:

Implementation	Weight storage	Compute next to storage	Steps
Standard-cell CIM, flip-flops	D flip-flops	AND gates + adder tree, standard cells	5.4, 5.5, 5.7
Standard-cell CIM, latches	Latches (about half the area of flip-flops)	as above	5.4, 5.7
Digital SRAM-CIM macro	Custom 6T SRAM bitcell + NOR/AND per cell	the 1-bit product is formed in the bitcell; adder tree in standard cells	5.2, 5.3, 5.6
Baseline	Same cell type as the CIM it is compared with, behind a read port	separate MAC array, fetch controller	5.5, 5.7

Why it matters, and what it does not claim.

- On the FPGA, CIM uses LUTs while the baseline's weights sit in BRAM and its arithmetic can sit in DSP slices, so an examiner can argue the FPGA favours or penalises one side. On the ASIC both are built from the same standard cells, so area, clock and energy compare like with like.
- On the FPGA the `n_macs` sweep stops when the part is full. On the ASIC it continues (Step 5.7): the limits become area, timing (adder-tree depth grows as log N) and power, which is the trade-off the scaling study shows.
- `BPC` (bits per cycle) trades area for throughput: `BPC = b` needs `b` adder trees per column and finishes a dot product in `8 / b` cycles. It is swept only on the ASIC.
- **Nothing is fabricated.** Every number is a tool estimate with its stage, corner and activity source stated.
- **A compiled SRAM is not CIM.** A memory compiler gives a black-box macro that reads one row per access, which is the baseline's dataflow. SRAM-CIM needs the custom bitcell array of Steps 5.2–5.3, with all 16 rows active every cycle and each cell's product on its own wire.

Lanes. Steps 5.1–5.3 do not depend on the model and run from Week 3 alongside Stage 1. Steps 5.4–5.7 need the RTL of Stage 3.

Results file. ASIC numbers go in `results/asic_runs.csv`, never in `runs.csv`: `date,commit,flow_stage,config,storage,n_macs,bpc,corner,clk_target_mhz,fmax_mhz,area_um2,area_storage_um2,area_compute_um2,power_mw,energy_per_inf_nj,activity_source,bitexact_pct,notes` (`flow_stage` is `synth` or `postlayout`; `activity_source` names the simulation the switching activity came from).

Step 5.1 — ASIC scope, tools and flow bring-up

Week 3–5 · 7 days · Needs: 0.1 · Lane: alongside Stage 1

Goal. A scripted, working Cadence flow from RTL to a placed-and-routed, verified netlist, and an agreed ASIC scope.

Do.

1. **Confirm the tools and PDK** from `docs/resources.md` (Step 0.1 questions 2–4). Digital flow: Genus, Innovus, Xcelium, Conformal, Joules (or Genus power), Quantus. Custom flow for SRAM-CIM: Virtuoso, Spectre, PVS or Assura (DRC/LVS), Quantus (extraction), and Liberate if available (cell characterization). PDK: GPDK045 with its standard-cell library (gsclib045) and its DRC/LVS/extraction decks. Write down which are confirmed.

2. **Record the scope as Decision D5** (template in "Decisions to record"): which of the three implementations are attempted, which flow stages (synthesis only, or through place-and-route), and the stop point for the SRAM-CIM path.
3. **Objectives addendum.** Stage 0 is not edited. Write `docs/objectives_asic.md` with the ASIC design targets and get Sir's approval:
    - the CIM tile and engine synthesize to the PDK's standard cells from the same RTL as the FPGA, with gate-level simulation bit-exact against the integer reference;
    - CIM and baseline are synthesized and placed-and-routed to the same library at the same clock target, and area, maximum clock and estimated energy per inference are reported for both;
    - a digital SRAM-CIM bitcell and a 16 × 16 × 8 array are designed, pass DRC and LVS, and are characterized by post-layout simulation;
    - (hypothesis, tested in Step 7.4) CIM needs less energy per inference than the baseline at equal throughput.
4. **Scripted flow** in `asic/` (no GUI for the digital flow): `asic/genus/synth.tcl` (RTL, SDC, library → netlist, area, timing and power reports), `asic/conformal/lec.do`, `asic/innovus/pnr.tcl` (floorplan, place, CTS, route, signoff timing, SDF), and `asic/joules/power.tcl`. A `make asic_synth`, `make asic_pnr` and `make asic_power` target each, taking `TOP` and `CONFIG`.
5. **Hello world:** an 8-bit counter through the whole flow: synthesis, LEC, gate-level simulation in Xcelium, place-and-route, post-layout simulation with SDF.
6. **Draw** `docs/diagrams/asic_flow.drawio` (digital and custom paths).

Output. The confirmed tool list, D5, `docs/objectives_asic.md`, the `asic/` scripts, the counter run.

Review gate.

- ▢ A second member runs the counter through the whole flow from a clean checkout, using only the Makefile targets.
- ▢ LEC passes and the post-layout simulation of the counter is correct.
- ▢ Sir has approved the objectives addendum.

If it fails. If Cadence or the PDK is not available, switch to the open-source flow (Yosys, OpenROAD/OpenLane with the SkyWater sky130 PDK; Magic, Netgen and ngspice for the custom path) and record that in D5. The steps below stay the same; only the tool names change.

LLM help. Sonnet drafts the TCL. Verify every Genus, Innovus and Conformal command against the Cadence documentation; models invent tool options, and wrong ones fail silently or are ignored.

Step 5.2 — SRAM-CIM bitcell: schematic and layout

Week 6–10 · 18 days · Needs: 5.1 · Lane: alongside Stage 1

Goal. A digital compute bitcell that stores one weight bit and outputs `weight_bit AND activation_bit`, with verified schematic and layout.

Do.

1. **Schematic** in Virtuoso: a 6T SRAM cell plus a NOR- or AND-based compute gate (about 8–12 transistors). Ports: write wordline, bitline pair, compute wordline (the broadcast activation bit), and the product output. Write `docs/asic/bitcell.md` with the circuit, the transistor sizes and why.
2. **Spectre simulation** across the TT, SS and FF corners and −40, 27 and 125 °C: write 0 and 1, hold, compute with both activation values, and confirm the compute path never disturbs the stored bit. Report the static noise margin (hold and write), compute delay and energy per bit operation.
3. **Layout,** compact and designed to tile: shared wells and rails at the edges, a pitch that matches the array.
4. **DRC and LVS clean** with the PDK decks. **Parasitic extraction** (Quantus), then repeat the item-2 simulations on the extracted netlist.

5. **Draw** the bitcell schematic and layout for the report.

Output. The Virtuoso library (schematic, layout, extracted view), `docs/asic/bitcell.md` with the corner tables, DRC/LVS reports.

Review gate.

- ▢ Every corner and temperature passes write, hold and compute, before and after extraction.
- ▢ DRC and LVS are clean (reports attached).
- ▢ A non-author checks the schematic against the description in `bitcell.md`.
- ▢ Bitcell area in μm² is recorded in `asic_runs.csv` (`config = bitcell`).

If it fails. If the compute gate disturbs the cell, isolate it with a read buffer (8T-style decoupled read). If the layout won't tile, fix the pitch before drawing more cells.

LLM help. Opus for circuit reasoning (noise margin, disturb, sizing trade-offs). No model draws layout or sets PDK rules; every rule comes from the PDK documentation.

Step 5.3 — SRAM-CIM array and macro views

Week 10–13 · 14 days · Needs: 5.2; 3.1 for item 4

Goal. A 16-row × (16 × 8)-column SRAM-CIM array that drops into the tile in place of `weight_store` and the AND mask, with the views the digital flow needs.

Do.

1. **Generate the array** from the bitcell with a SKILL script or parameterized cell (rows, columns and bits per weight as parameters): 2,048 bitcells for one 16 × 16 int8 tile. Add the compute-wordline drivers (one per row), the write decoder and write drivers. Never place cells by hand.
2. **DRC and LVS** of the array. **Post-layout simulation** of write patterns and of compute with several activation patterns, checking every column's 8 outputs equal the gated weight.
3. **Macro views:**
   - a LEF abstract (size, pins, blockages) for Innovus;
   - a `.lib` with the timing and power of the compute path (activation in → product out) and of a write, characterized from Spectre (by hand or with Liberate);
   - a Verilog behavioural model `sram_cim_array.v` with the same ports and timing behaviour, used as `weight_store` with `STORAGE = "macro"`.
4. Run the tile testbench of Step 3.1 with `STORAGE = "macro"` (behavioural model).

Output. The array library, LEF, `.lib`, `sram_cim_array.v`, `docs/asic/macro.md` with area, compute delay and energy per array operation.

Review gate.

- ▢ Array DRC and LVS clean.
- ▢ Post-layout simulation shows the correct gated weights for every tested pattern.
- ▢ The tile testbench passes 100% with `STORAGE = "macro"`.
- ▢ The `.lib` delay of the compute path is within 10% of the post-layout Spectre delay it was taken from.

If it fails. If characterization is the bottleneck, characterize only the compute path and the write at the typical corner, and say so in `macro.md`.

LLM help. Sonnet drafts the SKILL generator; check every SKILL call against the Virtuoso documentation.

Step 5.4 — Standard-cell CIM tile: flip-flop and latch versions

Week 12–13 · 5 days · Needs: 3.1, 5.1

Goal. The CIM tile at the process node, with storage and compute area reported separately.

Do.

1. `make asic_synth TOP=cim_tile CONFIG=ff` and `CONFIG=latch`, for `BPC = 1` first. The SDC sets the clock target (start at 500 MHz for 45 nm and record what is achieved).
2. **Conformal LEC** between RTL and each netlist.
3. **Gate-level simulation** of each netlist with the Step 3.1 tile testbench (Xcelium).
4. Report area split into storage (the `weight_store` instances) and compute (AND, adder trees, accumulator), the critical path, and post-synthesis power for a stated activity.

Review gate.

- ▢ LEC passes for both netlists.
- ▢ Gate-level simulation passes the full tile testbench for both.
- ▢ Rows in `asic_runs.csv` for both, with storage and compute area separate.
- ▢ For the latch version, the timing report shows no unconstrained latch paths (or each is explained).

LLM help. Gemini reads the full Genus reports; Opus for any LEC failure.

Step 5.5 — Complete CIM engine and baseline at the node

Week 15–17 · 12 days · Needs: 3.3, 3.4, 5.4

Goal. Both complete engines through synthesis and place-and-route to the same library, bit-exact as netlists, with area, clock and energy per inference.

Do.

1. Synthesize `cim_engine` and `baseline_engine` (`STORAGE = ff`, default `n_macs`, `BPC = 1`) at the same clock target. LEC on both.
2. **Place and route** both in Innovus: floorplan, placement, clock tree, routing, signoff timing at the stated corner, SDF.
3. **Post-layout gate-level simulation** with SDF on 20 clips: class and logits bit-exact with `ref_int.py`, cycle count equal to RTL simulation.
4. **Switching activity** from that simulation (SAIF or VCD) → Joules (or Genus power) → average power and **energy per inference** = power × cycles / f_clk. Use the same clips for both engines.
5. Repeat items 1, 4 for the latch tile version at synthesis level.

Review gate.

- ▢ LEC passes and post-layout simulation is bit-exact on 20 clips for both engines.
- ▢ Both engines meet the same clock target, or the achieved frequency of each is recorded.
- ▢ The activity source and corner are recorded for every power number.
- ▢ Rows in `asic_runs.csv`, `flow_stage = postlayout`.

If it fails. If place-and-route of the full engine does not close, place-and-route the tile and report the engine at synthesis level, saying so.

LLM help. Gemini on the Innovus and Joules logs. Opus for timing-closure decisions.

Step 5.6 — SRAM-CIM tile: macro plus synthesized periphery

Week 15–17 · 9 days · Needs: 5.3, 5.4

Goal. A complete SRAM-CIM tile: the custom array from Step 5.3 with the adder trees, shift-accumulate and control synthesized from the same RTL, placed and routed together.

Do.

1. Synthesize the tile periphery with `STORAGE = "macro"` (the macro as a black box with its `.lib`).
2. Place the macro (LEF) and the periphery in Innovus; route; signoff timing; SDF.
3. Post-layout simulation of the tile testbench (macro behavioural model with the `.lib` timing).
4. Report area, clock and energy per tile operation, next to the flip-flop and latch tiles from Step 5.4.

Review gate.

- ☐ The tile testbench passes post-layout.
- ☐ Rows in `asic_runs.csv` with `storage = macro`.

If it fails. This is the **first step to cut** if Stage 5 runs late. The fallback stops at Step 5.3: report the macro's own area, delay and energy, and project the tile from them, clearly labelled as a projection.

Step 5.7 — ASIC scaling study

Week 17–18 · 7 days · Needs: 5.5, 2.4

Goal. Show how CIM and baseline scale when the FPGA's resource limit is gone.

Do.

1. Synthesis-level sweep of both engines over `n_macs ∈ {8, 16, 32, 64, 128, 256, 512, 1024}` and of the CIM engine over `BPC ∈ {1, 2, 4, 8}`, `STORAGE = ff` (and latch for a subset). Same library, same clock target, same activity source.
2. Place-and-route at least the smallest, the default and the largest configuration of each engine to check how far synthesis estimates drift from layout.
3. Plot against `n_macs` and `BPC`: area (storage and compute), achieved clock, cycles per inference, energy per inference and throughput per area.
4. For SRAM-CIM, project the storage area of each configuration from the bitcell area of Step 5.2, labelled as a projection.

Output. The sweep rows in `asic_runs.csv` and the plots (generated by script).

Review gate.

- ☐ Every configuration's cycle count equals the cycle model (Step 2.4).
- ☐ Synthesis vs place-and-route differences are reported for the checked points.
- ☐ Every plot names the library, corner, flow stage and activity source.

Step 5.8 — ASIC results write-up

Week 18–19 · 4 days · Needs: 5.5, 5.6 (or its fallback), 5.7

Do. Write `docs/asic_results.md` from `asic_runs.csv` only: the comparison table (SRAM-CIM vs flip-flop CIM vs latch CIM vs baseline at the same node and clock: area, storage area, maximum clock, energy per inference, throughput), the scaling plots, the bitcell and macro figures, and a limitations section (estimates not measurements; no SRAM memory compiler; characterization corners used).

Review gate.

- ☐ Every number traces to an `asic_runs.csv` row.
- ☐ GPT, prompted as a hostile examiner, is given the write-up. Every objection is answered.

> **Milestone D — "The engine at a process node."** Bit-exact netlists of both engines, the comparison table, the scaling study, and the SRAM-CIM bitcell and array.

Step 5.9 — Formal proof of the tile (stretch)

Week 18–19 · 8 days, stretch, only if JasperGold is confirmed and everything else is on track

Do. Write assertions that the tile's output equals the reference dot product. Prove them on a 4×4 tile first, then scale up as far as the tool allows. Even a reduced proof is a strong verification result.

STAGE 6 — HLS of the complete model (Weeks 14–18)

Lane. Owned by whoever has capacity once Stage 3 is under way. It implements the frozen model directly; there is no layer-1-only HLS version.

Step 6.1 — HLS of the complete model

Week 14–16 · 10 days (+4 for the optional sweep) · Needs: 1.12, 2.1

Do.

1. Implement the frozen model in Vitis HLS in plain C. Use the standard types from `stdint.h`: `int8_t` for weights and activations, `int32_t` for accumulators, and `int64_t` for the requantization product (a 32-bit accumulator times a 16-bit multiplier needs more than 32 bits). Port the loops from `ref_int.py` and `arith_spec.md`.
    - **Why not `ap_int`.** The current Vitis HLS guide documents its arbitrary-precision types (`ap_int`, `ap_fixed`) for C++ only, and the old C versions (`ap_cint.h`) can't be used with standard C compilers. So plain C means standard widths. Hand-written RTL can use exact widths; C will use 16 or 32 bits wherever the tool can't prove a narrower one is safe, so the HLS version may use more LUTs for that reason alone. Report that as a fact about the method ("plain C, standard integer types"), not as a flaw in the tool. If you want to separate the effect, add one extra C++ row using `ap_int` with exact widths.
    - Confirm that your installed Vitis HLS version accepts a `.c` top-level file, and keep to what C allows: no templates, no references (pass pointers or arrays), no `hls::stream` (C++ only; use arrays).
    - **Two C traps.** Right-shifting a negative signed integer is implementation-defined in C. The spec requires an arithmetic shift, so test negative values explicitly in the C simulation. And signed overflow is undefined behaviour, which is why the accumulator bound from `arith_spec.md` must hold.
2. The C testbench reads the frozen vectors of Step 1.12 (`docs/file_formats.md`).
3. Synthesize at the default `n_macs`-equivalent parallelism. **Optional:** a reuse-factor sweep (array partitioning and unrolling) to give the third reference point of Step 2.3 (cost per MAC-per-cycle).
4. Log engineering hours from day one, for the comparison.

Review gate.

- ☐ C simulation and C/RTL co-simulation are bit-exact on the 200 clips, including inputs that make the accumulator and the requantization product negative.
- ☐ The language and integer types are written in the comparison table (plain C, `stdint.h` types).
- ☐ The DSP count is reported for the whole design and compared with the RTL engine's whole-design number like for like. If multiplies were forced into logic, the directive used is recorded.

LLM help. Sonnet writes the HLS code. Verify every `#pragma HLS` against the Vitis HLS user guide (UG1399). Models invent pragmas, and a wrong pragma is silently ignored.

Step 6.2 — RTL vs HLS comparison write-up

Week 17–18 · 3 days · Needs: 6.1, 4.5

Do. Write it up from `runs.csv` and the hours log: LUT, FF, BRAM, DSP, f_max, cycles, II and engineering hours, hand-written RTL vs HLS, on the same board and clock target.

STAGE 7 — Measurement, then claims (Weeks 13–19)

Step 7.1 — Write the measurement protocol before measuring

Week 13 · 3 days · Needs: 2.3, 2.4

Do. Write `docs/measurement_protocol.md`, defining:

- **FPGA quantities.**
  - `L_inf` (inference latency): cycles from the last sample of the window being present in the input buffer to the class being valid at the output, × the clock period.
  - `II` (initiation interval): cycles between successive inference starts at steady state.
  - `L_e2e`: acquisition + preprocessing + `L_inf`, reported for realism; the claim is not built on it.
  - How each is measured: the on-chip counter, cross-checked against post-route simulation and an oscilloscope on a toggle pin. How many runs, and what is reported (median, plus min/max).
  - The board. Headline measurements are taken on the platform chosen in Decision D14. The other board is reported as a second platform where the design fits.
- **ASIC quantities** (estimates, Stage 5): area (post-layout, total and storage/compute), maximum clock (signoff STA at the stated corner), energy per inference (power from the power tool with switching activity from post-layout simulation of the stated clips, × cycles / f_clk), and which configurations get place-and-route.
- The configurations, clock targets and the rule that numbers from different targets are never compared with each other.

Review gate.

- ▢ Sir approves the protocol before any Stage 7 measurement is taken.

Step 7.2 — Run the FPGA measurements

Week 16–17 · 6 days · Needs: 7.1, 4.6

Do. Run every configuration on the board per the protocol. Everything goes into `runs.csv`. (The ASIC estimates come from Stage 5 and are already in `asic_runs.csv`.)

Step 7.3 — Optional microcontroller baseline

Week 18–19 · 8 days, stretch · Needs: 1.10

Do. Run the same quantized model on a Cortex-M4 board using a plain-C integer forward pass ported from `ref_int.py` and `arith_spec.md`. The C code from Step 6.1 is a good starting point because both are plain C. This is a better baseline than TFLite-Micro (C++, its own quantization scheme), because it computes exactly your arithmetic. Clearly label it in the report as a different class of device.

Step 7.4 — Compare predictions with measurements, then write claims

Week 18–19 · 4 days · Needs: 7.2, 3.5, 5.7

Do.

1. Plot predicted (Step 2.4) vs simulated (Step 3.5) vs measured (Step 7.2) cycles.
2. Plot measured FPGA cycles against `n_macs` for both engines, and the ASIC area, clock and energy against `n_macs` and `BPC` (Step 5.7). These are the plots that test the hypotheses.

3. Only now, write the claims. Each claim sentence states what was measured or estimated, under which configuration, on which target, and cites the `runs.csv` or `asic_runs.csv` rows it comes from.
4. If a hypothesis was not supported, or only partly, report exactly that. "The baseline saturated at 32 MACs with a 64-bit weight port; CIM continued scaling to 128" is a claim. "CIM is 5× faster" without the configuration is not.

Review gate.

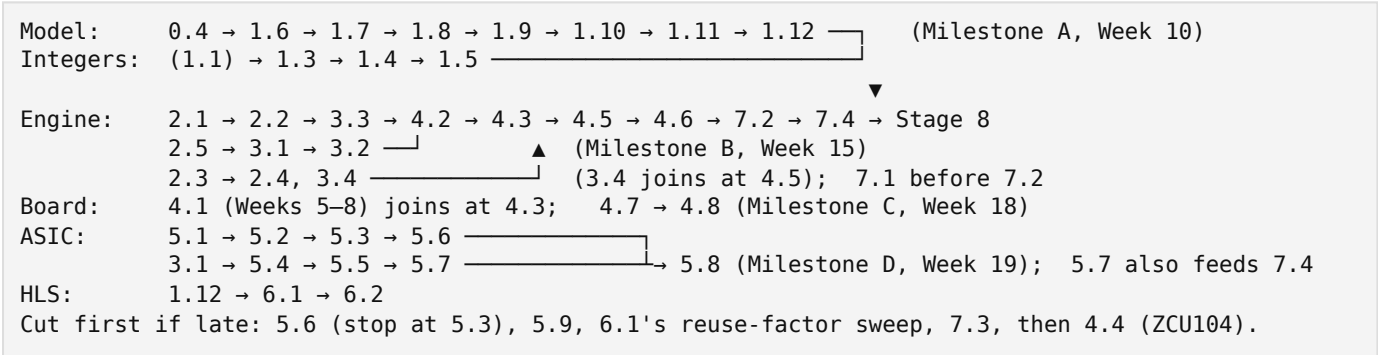
- [ ] Every number in the results chapters traces to a `runs.csv` or `asic_runs.csv` row.
- [ ] Every ASIC number is called an estimate and names its flow stage, corner and activity source.
- [ ] GPT, prompted as a hostile examiner, is given the claims and the protocol. Every objection is answered.

STAGE 8 — Demo polish, report, defence (Weeks 6–20)

Write the report from Week 6, not Week 16. Each spec document (`dataset_audit`, `arith_spec`, `mapping_spec`, `baseline_spec`, `rtl_rules`, `bitcell.md`, `macro.md`, `measurement_protocol`) is most of a methodology section already.

Step	Week	Effort (d)	What
8.0	6–16	6	Methodology chapters drafted from each spec document as it passes its gate (about half a day per spec); the model chapter (data, training, quantization, test results) in Week 11, from Milestone A.
8.1	17–18	4	Results chapters generated from `runs.csv` and `asic_runs.csv`. Figures produced by scripts.
8.2	18	4	Full draft. Each member reviews a chapter they did not write.
8.3	18–19	3	Structural review (Opus) and language polish (GLM or Sonnet).
8.4	19	2	Mock defence. Each member presents a part they did not build, and someone outside the team attacks it.
8.5	19–20	5	Fixes, slides, demo rehearsal × 3.
8.6	20	1	Submit and defend.
—	21–24	—	Buffer. Nothing is planned here. This is what absorbs a slip; the end of March is the hard limit.

The critical path, at a glance



Effort check

One effort day = 6 focused hours. Remaining work from Week 3 (Stage 0 and Step 1.2 are done):

Stage	Core (person-days)	Stretch
1 Complete model	56	
2 Architecture	27	
3 RTL	51	
4 FPGA, boards, demo	48	
5 ASIC	76	8 (5.9)
6 HLS	13	4 (reuse sweep)
7 Measurement and claims	13	8 (7.3)

Stage	Core (person-days)	Stretch
8 Report and defence	25	
Total	**309**	**20**

Four people at 25 hours a week give about 300 person-days in Weeks 3–20. **The core plan uses about 103% of that capacity, so it fits only by using part of the Weeks 21–24 buffer (about 67 person-days).** The remaining core work in v3 was about 245 person-days; v4's is about 309 (+64). The ASIC milestone adds about 76 and the new Python engine model, RTL rules (Steps 2.2, 2.5) and HLS of the complete model about 12; building the hardware once (no layer-1 engine, synthesis or board run) and dropping the fallback workload save about 24. The table sums the effort in the step headers, which is person-days of work, not calendar time.

What to do about it:

1. **Cut Step 5.6 at the first sign of slip** (saves about 9 days, bringing the plan to about 100%). The SRAM-CIM result then stops at the verified bitcell and array (Steps 5.2–5.3) with a projected tile, which is still a complete ASIC milestone.
2. **Protect Weeks 10–17.** Stage 3, the board steps and Stage 5 overlap there and run at about 120% of capacity. Move coursework away from those weeks.
3. **Stretch items only if ahead:** 5.9, the HLS reuse sweep, 7.3.
4. If the real hours are nearer 20 per week, cut 5.6 and 4.4 at once and plan to use the buffer.
5. **If Milestone A (Week 10) is more than 2 weeks late,** cut 5.6, 4.4 and all stretch items immediately and limit the ASIC scaling study (Step 5.7) to synthesis level.

The SRAM-CIM bitcell (Step 5.2) needs someone with transistor-level layout experience. If nobody on the team has it, decide in Step 5.1 whether to learn it (add about 5 days) or limit Stage 5 to the standard-cell implementations.

Decisions to record

The tracker's Decisions sheet holds D1–D18 with these IDs; `docs/decisions.md` holds the full write-ups. v4 keeps every existing ID and meaning, changes the status of some, adds D19–D20 (already written up in `docs/decisions.md`) and adds D21–D24:

ID	Decision	Step (v4)	Status in v4
D1–D3, D7–D10, D13	as in the tracker	moved with their steps (see the numbering table)	unchanged
D4	Class definition after the audit: `pistol_9mm`, `rifle_556_223`, `revolver_38spl`, `shotgun_12ga`, `background`	0.4	**decided** (reasons in `docs/dataset_audit.md`)
D5	ASIC scope: tools, PDK, which implementations, SRAM-CIM stop point (was "Cadence scope")	5.1	open, Week 4
D6	ADC sample rate, bit depth and window length	1.7	open
D11	Primary workload if session-disjoint accuracy is too low	0.4	**decided:** no fallback workload; Google Speech Commands not downloaded (`docs/dataset_audit.md` §6)
D12	Go/no-go for Genus synthesis and the DRAM-bound baseline	—	**superseded** by D21: ASIC is core; the DRAM-bound baseline is optional and unscheduled

ID	Decision	Step (v4)	Status in v4
D14	FPGA measurement platform	4.4	open (now after the complete engine runs on both boards)
D15	ZCU104 host interface (fabric UART or processor system)	4.4	open
D16	Microphone (external I²S)	4.7	open
D17	Scope of the "no DSP" statement	0.2	as in the tracker
D18	Weights for the early hardware tests	1.1	**superseded:** no early hardware tests; the quick model's weights are used only in software
D19	GAP sum requantized to int8 before `dense`	1.2	proposed, in `docs/decisions.md` (add to the tracker)
D20	How the dense output is compared in argmax (requantize each class onto one logit scale)	1.2	proposed, in `docs/decisions.md` (add to the tracker)
D21	Complete model first, hardware built once for the frozen model; ASIC as a full milestone	this runbook	new, to record
D22	Window placement (recommended: 0.5 s input, 0.40 s of real audio, the last 0.10 s zeroed for every source); the length itself is D6	1.6	new, open
D23	Final test sessions (recommended: every clip with `session_id` `c3gd_nj_farm_2` or `edge-day2`, background included, plus 15% of the remaining background sessions)	1.6	new, open
D24	`BPC` addendum to `arith_spec.md` §4 and the `STORAGE` parameter	2.5	new, open

Which LLM for what

Job	Model	Why
Writing code from a clear spec (RTL, testbenches, Python, TCL, plotting)	Sonnet	Most of your volume. Mechanical work with a clear specification.
Spec documents, arithmetic and quantization math, architecture and baseline fairness, circuit reasoning for the bitcell, bugs that survive an hour	Opus	Where a wrong answer costs a rework cycle.
A bug Opus has failed on twice, or a costly one-way decision	Fable	Rarely; probably 3–4 times all year.
Huge inputs: full Vivado, Genus, Innovus and Joules logs, timing reports, datasheets, PDK documentation, several papers at once	Gemini	Long context. Use it to extract and summarize, not to design.
Attacking your claims, specs, baseline and ASIC write-up as a hostile examiner	GPT	A model outside the conversation finds what you've stopped seeing.
Boilerplate, docstrings, LaTeX formatting, language polish	GLM	Cheap and adequate for non-critical text.

Rules:

1. The integer reference and the RTL testbench are written in separate conversations, from the spec document only. Never let one model see the other's code while writing.
2. No LLM writes a number into results. Every number comes from `runs.csv` or `asic_runs.csv`.

3. Verify every vendor-specific detail against the official documentation: Vivado TCL options, XDC constraints, `#pragma HLS`, pin assignments, Genus/Innovus/Conformal commands, SKILL calls, PDK design rules. Models invent these, and they fail silently.
4. Clock-domain-crossing logic comes from a proven template, never written fresh by a model.
5. No model draws layout or chooses transistor sizes without a Spectre simulation that confirms them.
6. Start a fresh conversation for each step, and paste in the relevant spec. Long threads get slower, costlier and less accurate.

Data sources (verify before citing)

Gunfire (used in `Dataset-Final/`):

- R. Kabealo et al., "A multi-firearm, multi-orientation audio dataset of gunshots," *Data in Brief*, vol. 48, 109091, 2023. Dataset: Zenodo DOI 10.5281/zenodo.7004819 (CC BY 4.0).
- S. Gurny, R. Quinn, "Descriptor: Certus Caliber Classification Gunshot Dataset (C3GD)," arXiv:2606.18135, 2026. Dataset v2: Zenodo DOI 10.5281/zenodo.22286299 (CC BY 4.0), 12,112 clips, 45 firearms, 18 calibers, 4 sessions.
- Cadre Research Labs (R. Lilien), "Development of Computational Methods for the Audio Analysis of Gunshots," NIJ grant 2016-DN-BX-0183, final report NCJ 252947, 2018. Test-firing recordings obtained by registration; terms of use to be confirmed before publishing Cadre audio.

Background:

- J.-W. Kim, C. Yoon, H.-Y. Jung, "A Military Audio Dataset for Situational Awareness and Surveillance," *Scientific Data*, 2024 (MAD).
- J. Salamon, C. Jacoby, J. P. Bello, "A Dataset and Taxonomy for Urban Sound Research," ACM Multimedia, 2014 (UrbanSound8K, CC BY-NC 3.0).
- K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," ACM Multimedia, 2015 (ESC-50, CC BY-NC 3.0).

Excluded: the BGG / "Enemy Spotted" in-game gunshot dataset (arXiv:2210.05917) is video-game audio. Copies of it were found inside the supplied `GUN_DATASET` and removed (`docs/dataset_audit.md` §2.5). Never use it as test data.

Related work: "Deciphering GunType Hierarchy through Acoustic Analysis of Gunshot Recordings," arXiv:2506.20609 (five broad gun-type categories).

ASIC:

- Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm² All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications," ISSCC 2021. The reference for the digital, bit-serial SRAM-CIM dataflow of Steps 5.2–5.3.
- M. R. Guthaus et al., "OpenRAM: An Open-Source Memory Compiler," ICCAD 2016. Only if the open-source fallback of Step 5.1 is used.
- Cadence GPDK045 documentation (from the PDK installation).

Step numbering, old to new

For updating the tracker. Stage 0 and Step 1.2 keep their numbers.

v3 step	v4 step	Change
0.1–0.4	0.1–0.4	none (done)

v3 step	v4 step	Change
1.1 Quick real-shaped model	1.1	windows from `Dataset-Final`; per-layer table must match `arith_spec.md` §5.1
1.2 Arithmetic spec	1.2	none (done)
1.3 Integer reference	1.3	independent versions of `conv1`, `conv2`, `dense`; argmax tie test
1.4 Tile model	1.4	adds `BPC`
1.5 Test vectors	1.5	every layer from the start
1.6 CIM tile RTL	3.1	after the model freeze; `BPC` and `STORAGE` parameters
1.7 Board bring-up, UART, tile on board	4.1	stand-in engine instead of the tile on the board
1.8 Layer-1 engine	3.2	merged with v3 3.4: all blocks for the full network
1.9 Synthesize layer 1	4.2	complete engine
1.10 Layer 1 on Nexys (Milestone A)	4.3	complete engine (Milestone B)
1.11 Layer 1 on ZCU104	4.4	complete engine
2.1 Preprocessing and splits	1.6	window placement D22, test sessions D23, field-mic background ≥ 15%
2.2 Fallback workload	—	removed (D11)
2.3 ADC sweep	1.7	adds the 0.4 s window
2.4 Float models	1.8	macro-F1, per-source results
2.5 Model selection gate	1.9	freezes the architecture
2.6 Quantize	1.10	
2.7 Trained model through `ref_int`	1.11	the held-out test is run here, once
2.8 Real test vectors	1.12	model freeze, tag `model-v1` (Milestone A)
3.1 Mapping spec	2.1	
—	2.2	new: Python engine model
3.2 Baseline spec	2.3	ASIC baseline uses the same storage cells
3.3 Cycle model	2.4	adds `BPC`, larger `n_macs`
—	2.5	new: target-neutral RTL rules, `BPC` addendum
3.4 Remaining engine blocks	3.2	merged
3.5 Integrate full engine (sim + board)	3.3	simulation only; the board run is 4.3
3.6 Baseline RTL	3.4	
3.7 Sweep in simulation	3.5	adds `BPC`
4.1 Build all configurations	4.5	
4.2 Full test set on boards	4.6	
4.3 Microphone path	4.7	detector checked against `detector.py`
4.4 Live demo (Milestone B)	4.8	Milestone C; false-alarm rate
5.1 Decide Cadence scope	5.1	flow bring-up, D5, objectives addendum
5.2 HLS of layer 1	6.1	complete model

v3 step	v4 step	Change
5.3 Genus synthesis of the tile (stretch)	5.4	core; flip-flop and latch versions
5.4 Formal proof (stretch)	5.9	still stretch
5.5 HLS of the real model (stretch)	6.1	merged, core
5.6 RTL vs HLS write-up	6.2	
—	5.2, 5.3, 5.5, 5.6, 5.7, 5.8	new: SRAM-CIM bitcell and array, engines at the node, SRAM-CIM tile, scaling study, ASIC write-up (Milestone D)
6.1 Measurement protocol	7.1	adds ASIC definitions
6.2 Run measurements	7.2	
6.3 MCU baseline (stretch)	7.3	
6.4 Predictions vs measurements, claims	7.4	adds ASIC plots
7.0–7.6 Report and defence	8.0–8.6	same steps, renumbered; results from `runs.csv` and `asic_runs.csv`

Monday morning (Week 3, from Monday 26 Oct 2026)

1. Record D4 (decided), D11 (decided), D12 and D18 (superseded) and D21 (new) in `docs/decisions.md` (D19 and D20 are already there), and update the tracker to v4 (steps, tasks, milestones, decisions).
2. **Model lane:** Step 1.1 (`cnn1d.py` and the per-layer table) on Monday; Step 1.6 (detector, Decisions D22 and D23) from Tuesday.
3. **Integer lane:** Step 1.3 (`quantize.py`, `ref_int.py`) on the quick model, and Step 1.4 (tile model).
4. **ASIC lane:** Step 5.1. Confirm the Cadence tools and PDK and run the 8-bit counter through the flow by the end of Week 4. Find out who on the team has done transistor-level layout.
5. **Board lane:** Step 4.1 starts in Week 5.

For whoever runs individual steps from this plan: Sonnet for most steps, with the relevant spec pasted into a fresh conversation. Opus for Steps 1.3, 1.6, 1.10, 2.1, 2.3, 2.5, 5.2 and any bug older than an hour. Extended thinking only for Steps 1.10, 2.3 and 5.2 (quantization scaling, baseline fairness, bitcell design).